

The Sentinel System

A Multi-Layered Passive Monitoring Framework for Safeguarding Ambient Clinical AI Scribes

Nathan Mani

IX, Imperial College London
nathan.mani@imperial.ac.UK

September 1, 2026

Abstract

The integration of Artificial Intelligence as a Medical Device (AIaMD), specifically Ambient Voice Technology (AVT), offers significant operational efficiencies in healthcare but introduces probabilistic risks such as clinical hallucinations and omissions. Current post-market surveillance frameworks, such as the UK's Yellow Card Scheme, are structurally reactive and rely heavily on the manual reporting of overworked clinicians, creating a safety vacuum exacerbated by automation bias. To address this gap, this paper evaluates different deterministic computationally cheap methods, including rule based Natural Language Processing, Kernel Density Estimation, and trained alignment models, for the real time detection of factual inconsistencies between clinical audio transcripts and generated SOAP notes.

Empirical testing on the augmented PriMock57 dataset revealed that cross document verification struggles due to a register mismatch between spoken clinical dialogue and structured written summaries. However, targeting the high risk assets in the summaries (via a module called NoteExtractor) successfully bypassed this mismatch, achieving a 90.6% F1 score in identifying high risk clinical entities such as medications, negations, and inferences.

Building upon these findings, this paper introduces The Sentinel System: a functional, vendor-agnostic Proof of Concept delivered as a client-side browser extension and local API. This middleware passively monitors AI edits to capture an auditable telemetry trail of model drift without adding clinical overhead, while highlighting critical entities to introduce cognitive friction and dismantle the dangerous "click-through" verification loop. Ultimately, this paper proposes a policy white paper advocating for the MHRA to mandate continuous, deterministic passive telemetry, thereby modernizing clinical AI governance to match the safety standards of traditional medical software.

Acknowledgement

I would like to thank Brendan C Delaney my Imperial supervisor as well as Domenic DiFrancesco and Rozelle Kane from the EMBLA team for guiding me and helping with my various questions. I also thank Davide Locatelli for access to the mock AVT and more.

Contents

1	Introduction	5
2	Landscape of Clinical AIaMD	6
2.1	AI as a medical Device	6
2.1.1	AI benefits and concerns	7
2.2	Probabilistic software in healthcare	9
3	Operational Risks of AI in healthcare	9
3.1	Taxonomy of AI-Induced Risks in Clinical Environments	9
3.1.1	Statistical and Distributional Risks (Algorithmic Decay)	9
3.1.2	Socio-Technical and Cognitive Risks (Human-AI Interaction)	10
3.1.3	Generative and LLM-Specific Vulnerabilities	12
3.1.4	Architectural + Systemic Blind Spots	12
4	Current Evaluation Paradigms and Reporting Gaps	13
4.1	Pre-Deployment Testing and Benchmarking Limitations	13
4.2	Post-Market Surveillance and the Yellow Card Framework	14
4.2.1	The Yellow Card Scheme	15
4.2.2	Manufacturer's Online Reporting Environment (MORE)	15
4.2.3	Routine Clinical Auditing	15
4.2.4	Continuous Real Time Monitoring	16
5	Modernizing Surveillance: Integration and Analytics	16
6	The Proposed Solution: Targeted Evaluation via Ambient AI Scribes	17
6.1	AVT Embedded Risk Mitigation	17
6.2	AVT System Architecture	18
6.3	What this work adds to clinicians	18
7	This Paper's Objectives	19
8	Test data	20
8.1	PriMock57 Corpus (babylonhealth/PriMock57)	20
8.1.1	How the Dataset Was Augmented	21
9	Determinism in a non-deterministic space	24
9.1	Natural Language Processing	24
9.1.1	spaCy as a Foundation	25
9.1.2	scispaCy	25
9.1.3	NegspaCy	25
9.1.4	MedspaCy	26
9.1.5	UMLS Concept Mapping: MetaMap and QuickUMLS	26
9.2	Proposed Use of NLP for SOAP checker	26

9.2.1	Selecting a condenser: cosine and ROUGE-1 evaluation	26
9.2.2	The medspaCy based checker	28
9.2.3	Error Analysis & Failure Modes	28
9.3	KDE-Based Semantic Density Checking (EmbedKDECheck)	29
9.3.1	How EmbedKDECheck works	29
9.3.2	Why This Fits an Ambient AI Post-Market Check	30
9.3.3	Proposed Use: Bidirectional, Per-Sentence Density Checking	30
9.3.4	Results	31
9.3.5	Error Analysis & Failure Modes	31
9.3.6	Possible Fixes for Further Research for using KDE in this context	32
9.4	AlignScore: Trained Alignment Model	32
9.4.1	How AlignScore works	33
9.4.2	Why This Fits an Ambient AI Post-Market Check	33
9.4.3	Proposed Use: AlignScore as a Per-Sentence Hallucination Check	34
9.4.4	Results	35
9.4.5	Error Analysis & Failure Modes	36
9.4.6	Possible Fixes for Further Research for using AlignScore in this context	36
9.5	SUMMAC	37
9.5.1	How SummaC Works	37
9.5.2	Why This Is a Harder Fit for an Ambient AI Check	37
9.6	NoteExtractor: A Lightweight Structured-Summary Layer	38
9.6.1	How NoteExtractor Works	38
9.6.2	Results	38
9.6.3	Error Analysis & Failure Modes	38
10	Results	40
10.1	In Documents Tag Contribution	41
10.2	Cross-Document Verification Can Still Improve	41
11	Example Market deployment	41
11.1	What the Prototype Does	42
11.2	Why a Browser Extension Is the Right Delivery Mechanism	43
11.2.1	Closing the Loop	44
12	Method Limitations	45
13	Conclusion	46
14	Legal, Social, Ethical, and Professional Issues (LSEPI)	46
14.1	Ethical Review and Data Governance	46
14.2	Patient Consent and Acoustic Surveillance	46
14.3	Medico-Legal Liability and Duty of Care	46
14.4	Health Equity and Algorithmic Bias	47
15	Policy Recommendation: A White Paper for AIaMD Governance	47
16	Future Work	47
17	Appendix A: Code	52
17.1	Directory tree	52
17.2	Files under Project	53
17.2.1	run_checker_modules.py	53
17.2.2	run_condensers.py	53

17.2.3	run_note_extractor.py	53
17.3	Environment setup	53
17.4	Running things	54
17.4.1	Full-dataset evaluation runs	54
18	Appendix B : Running the google extension	55
18.1	Prerequisites	55
18.2	Running the server	55
18.3	Loading the extension into Chrome	55
18.4	How the extension connects to the server	57
18.5	Architecture at a glance	57
18.6	Using the extension	57
18.7	Known caveats	58

1 Introduction

The medical field is notoriously sceptical when adopting new technologies to ensure that leveraging the benefits doesn't affect patient safety. As such, multiple schemes have been instantiated to provide post market surveillance of these technologies, not to criticise the medical products but help improve their reliability and robustness. AI presents a new challenge to the industry, where its benefits improve the 'quadruple aim' of healthcare; population health, patient's experience of care, caregiver experience and reduce cost of care, (1) however it also presents its own unique risks in terms of hallucinations, deskilling and omissions (2) which, when overlooked, can cause detrimental downstream effects to patient well being (3) . As with any new technology, regulatory bodies step in to help monitor and protect the public from casualties due to mass adoption of unproven products.

Driven by rapid technological advancement in the 2000's, Software as a Medical Device (SaMD) was explicitly classified in 2013 to help regulators manage software native technologies in medicine, ensuring that a deterministic input "X" would consistently yield a predictable output "Y" (4) . This managed structure kept risks and updates tightly controlled. However, the inclusion of non deterministic, evolving software required an expansion of this framework; in 2021, Artificial Intelligence as a Medical Device (AIaMD) was instantiated to govern the integration of adaptive, probabilistic algorithms safely into the medical domain (5) (2) .

Given the inherently probabilistic nature of machine learning, system degradation over time is inevitable. These degradation vulnerabilities can be fundamentally divided into data driven and model driven problems. Data drift occurs in environmental shifts, such as patient variations in the real world deployment no longer reflecting the initial training distribution used for the AI model (6) . This can result in incorrect outputs from the AIaMD; if unnoticed can cause downstream effects. Concurrently, model drift are compromises in internal dynamics, manifesting as calibration decay or performance drops in the AI (7) . Since AI does not yield an exact, deterministic "Y" output to an "X" input, unexpected behavioural shifts occur despite rigorous pre deployment testing.

To mitigate these errors, regulations mandate a "human in the loop" framework (5) . However, the human component is highly susceptible to automation bias, where stressed, overworked practitioners over rely on the AI's output and blindly accept a flawed generation. Conversely, some practitioners find that the cognitive load of meticulously verifying an AI's work to ensure final accountability creates an entirely new documentation overhead (8) (9) .

This tension between efficiency and safety is acutely visible in Ambient Voice Technology (AVT) used for curating SOAP notes from clinic conversations, a common use of AIaMD, whose errors primarily manifest as clinical hallucinations, where the model invents medical facts, or safety critical omissions, for example where the AVT fails to record dangerous contradictions or allergies. In high stakes medical settings, these silent errors carry catastrophic clinical risks.

While AIaMD is covered under updated post market surveillance mandates , such as the Yellow Card scheme , expecting time pressured clinicians to manually catch early signs of semantic degradation or subtle algorithmic drift is a dangerous assumption given the already heavy workload healthcare professionals are under (10) . Current infrastructure relies heavily on reactive, manual incident logging after an adverse event has already occurred. This creates a severe communication divide and a safety vacuum between AIaMD Manufacturers and real time clinical reality.

To address this gap, this paper introduces different real-time evaluation methods as well as an example passive monitoring framework designed to sit directly within the clinical pipeline. By executing real time, deterministic anomaly detection and rule based semantic checks, this system aims to capture and alert practitioners of clinical hallucinations, safety critical omissions and possible drift in ambient voice tools, before they propagate into the electronic health record (EHR). This system provides NHS healthcare professionals with an automated, auditable safe-

guard to ensure long term clinical safety and operational vigilance. The proposed framework hopes to set an example of AI monitoring across AIaMD applications.

2 Landscape of Clinical AIaMD

2.1 AI as a medical Device

The evolution of AI presents an opportunity to transform the healthcare landscape as much or as more than the previous era of rule based, deterministic technology. To manage the previous technological progression, regulators classified SaMD in 2013 to govern deterministic programs where a specific input X consistently yielded a predictable output Y. However AI is natively non-deterministic and requires a broader framework; in 2021 AIaMD was instantiated to safely govern the integration of adaptive, non-deterministic algorithms into clinical workflows. SaMD is defined for software external from the workings of embedded technology. (4) . The legislation established risk categorization matrix based on the criticality of the patient's condition (Critical, Serious, Non-serious) and how heavily the healthcare provider relies on the software's output. The matrix classes SaMD into categories with its own regulatory checklist to meet, with the more critical and more relied having the most checks. However the underlying assumption of the framework was that the software was static.

		Significance of Information provided by the MDSW to a healthcare situation related to diagnosis/therapy		
State of Healthcare situation or patient condition		High Treat or diagnose ~ IMDRF 5.1.1	Medium Drives clinical management ~ IMDRF 5.1.2	Low Informs clinical management (everything else)
	Critical situation or patient condition ~ IMDRF 5.2.1	Class III Category IV.i	Class IIb Category III.i	Class IIa Category II.i
	Serious situation or patient condition ~ IMDRF 5.2.2	Class IIb Category III.ii	Class IIa Category II.ii	Class IIa Category I.ii
	Non-serious situation or patient condition (everything else)	Class IIa Category II.iii	Class IIa Category I.iii	Class IIa Category I.i

Figure 1: The IMDRF SaMD Risk Categorization Matrix. This framework classifies medical software into four categories based on the severity of the clinical situation (Critical, Serious, or Non-Serious) and the impact of the software's output (Inform, Drive, or Treat/Diagnose). Category IV represents the highest risk profile, requiring the most rigorous regulatory scrutiny for deployment.

With new AI medical products breaking that “static” mould through dynamic systems that continuously adapt and optimise algorithms to effectively ingest real-world data, resulting in non-deterministic outputs, the MHRA, alongside the FDA and Health Canada, established the guideline: the pillars of Good Machine Learning Practice (GMLP) (11) . This acts as an addition to SaMD legislation and is aimed at keeping practices incorporating AI standardised and more traceable for regulatory purposes.

GMLP provides ten foundational principles that merge medical device quality management systems (QMS) with agile data science practices. The framework enforces strict oversight across the total product lifecycle (TPLC), specifically mandating data representativeness to mitigate

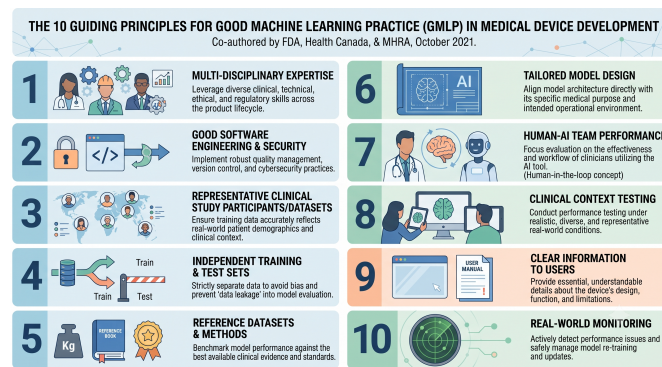


Figure 2: The 10 Guiding Principles for Good Machine Learning Practice (GMLP), established by international regulators to govern the total product lifecycle of AI-enabled medical devices. The framework emphasizes cross-functional expertise, data independence, human-AI team workflows, and continuous real-world performance tracking to eliminate safety vacuums post-deployment.

algorithmic bias, the absolute separation of training and testing datasets, and structured post-market monitoring to track model performance in real-world clinical environments. although not enforced this guideline is used by regulators to do checks on new AIaMD that enter the market (12) .

2.1.1 AI benefits and concerns

In the UK, AI has established use in clinical imaging and administrative automation, though its widespread systemic integration across the National Health Service (NHS) is established it is still in its infancy. With reported 67 percent of AI adoption in the UK being in its early stages (13) , the top 3 factors for the decision for AI integration complement the "quadruple aim" of healthcare" (1) . Figure 3a.

On an operational level, empirical data demonstrates substantial performance enhancements, particularly in mitigating workforce burdens; ambient clinical documentation and automated administrative systems have been shown to save frontline staff an average of 43 minutes per day (14) . Furthermore, predictive data analytics applied to outpatient scheduling logistics have achieved up to a 30 percent reduction in missed appointments ("Did Not Attends"). This dual tracked utility of diagnostic triage and back office optimization underscores the focal point of contemporary oversight initiatives, such as the Medicines and Healthcare products Regulatory Agency's (MHRA) "AI airlock" sandbox ,which seek to safely validate these models while addressing clinician concerns regarding algorithmic drift and legal liability.

Currently, 28% of general practitioners (GPs) in the UK actively utilize AI within their daily practice (13) . Far from a uniform distribution of clinical tasks, deployment is dominated by clinical recording utilities; clinical note-taking and documentation represent 57% of all active AI applications in GPs, driven primarily by AVT, followed secondarily by tools supporting professional development (13) . This concentration aligns with broader healthcare expectations, where 31% of the public and 32% of medical staff identify administrative automation as the premier utility for AI in healthcare, supported by a 61% public approval rate for automating administrative workflows (13) . However, systemic optimism remains tempered, with an average of only 30% of individuals across all age demographics believing that AI will fundamentally improve overall healthcare delivery (13) Figure 3b .

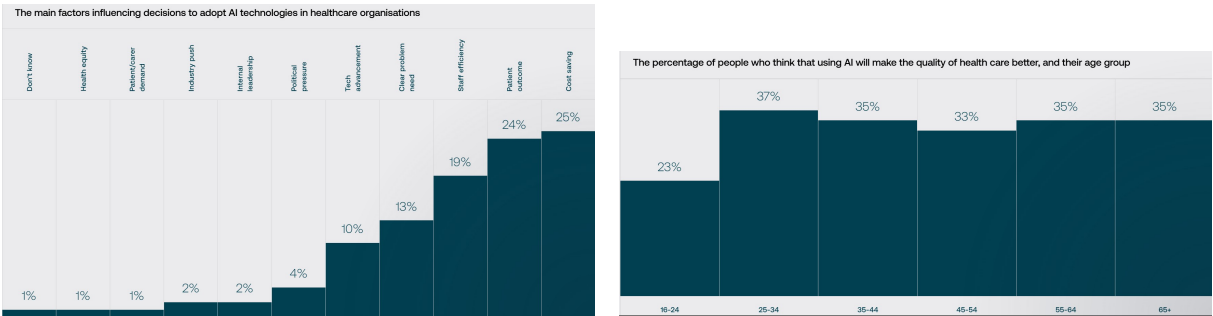
This caution highlights structural anxieties regarding clinical safety, workforce readiness, and the deficit in live operational safeguards. The primary justification for continuous, automated AI post market monitoring lies at the intersection of severe workforce strain and inadequate training

infrastructure for using AI. While 87% of healthcare professionals acknowledge that specialized training is mandatory to safely operate AI tools, only 47% report having the available time to complete extra training amidst heavy clinical workloads, with 34% feeling that their teams possess the necessary competence to utilize the technology safely (13)

Consequently, these deployment gaps invite severe operational risks. Approximately 30% of the public express concern that over extended clinicians will fail to question flawed AI generations, a clear societal recognition of the threat of automation bias (13) . This anxiety of algorithmic errors is mirrored by the medical workforce itself, shared by 83% of non AI users and 69% of active AI using clinicians (13) . These fears stem from a recognized regulatory vacuum after an AIaMD tool goes live: 88% of non users and 78% of active AI using GPs cite a lack of ongoing oversight regarding these deployed systems (13) . When a majority of active users acknowledge that their tools lack systematic oversight, relying solely on static, pre-market approvals this raises concern for a possible safety risk. Continuous monitoring is required to bridge the gap between initial software clearance and post deployment, clinical reality.

These insights provide the rationale for this work’s focus on the post-market surveillance of AVT. Because AVT represents the most heavily adopted clinical AI application in primary care (57%) (13) , it constitutes the single largest daily exposure for non-deterministic, semantic errors within the clinical pipeline.

When a time-pressured practitioner, who lacks the operational bandwidth to undergo rigorous tool training , is paired with an adaptive, unmonitored documentation tool, the likelihood of unnoticed clinical hallucination or safety critical omissions becomes more likely. Given that 69% of active AI users openly fear that these applications will induce clinical errors (13) , expecting busy frontline staff to act as the primary firewall against subtle text degradation can be considered a dangerous assumption. By targeting AVT with an automated, pro-active, post-market monitoring framework, this system safeguards the specific clinical intersection where the highest volume of daily AI utilization crosses the deficit in human verification capacity, in an attempt to reduce automation bias.



(a) Institutional drivers for AI adoption. (b) Healthcare optimism by age demographic. (13)

Figure 3: There is a systemic friction between institutional deployment drivers and public skepticism. While organizational adoption of AI is compelled by operational cost savings (25%) and staff efficiency strains (19%) Figure 3a, public confidence regarding AI’s net benefit to healthcare quality remains stagnant, averaging roughly 30% across all age demographics Figure 3b. This divergence underscores the necessity of continuous post-market surveillance; healthcare systems are rapidly integrating AI automation to manage workforce burdens, yet are doing so without ongoing, live oversight required to appease widespread clinical and public safety anxieties.

2.2 Probabilistic software in healthcare

The paradigm shift from traditional, rule-based SaMD to AI architectures introduces technical and clinical vulnerabilities rooted in the transition from deterministic to probabilistic logic. Traditional medical software operates under explicit static code structures, where system anomalies typically manifest as, trackable execution crashes or clear runtime exceptions. Conversely, machine learning models operate probabilistically, calculating statistical associations across n -dimensional feature spaces. When exposed to evolving real world environments, these systems are highly susceptible to "silent failures". A phenomenon where the algorithm continuously output clinically erroneous predictions with high statistical confidence without triggering standard software alerts or exceptions (15). This vulnerability is driven primarily by two distinct, interconnected phenomena: data drift and model drift.

- Data drift (frequently manifesting as covariate shift) occurs when the distribution of the live, operational input data diverges from the historical distribution of the training dataset. In clinical settings, this is often catalyzed by shift in institutional patient demographics, modifications to EHR data entry fields, or changes in diagnostic hardware and imaging vendors (16) (17).
- Model drift (often associated with concept drift) represents the internal calibration decay of the model's predictive performance over time. This happens when the underlying statistical relationship between the input clinical features and the targeted medical labels degrades, usually due to evolving clinical guidelines, changing disease prevalence, or updated therapeutic standards.

The downstream clinical consequences of these unmonitored probabilistic shifts are severe, directly threatening patient data integrity and fracturing treatment planning workflows. Because if a drifted model continues to present incorrect outputs with high confidence calibration scores, it exploits clinician's automation bias. Healthcare providers may unknowingly integrate flawed algorithmic recommendations into medical pathways, such as in oncology staging, cardiac risk stratification, or acute triage decisions. Furthermore, because macro level performance averages frequently mask acute degradation within localized patient demographics, silent model failures systematically exacerbate healthcare inequities. This drives misdiagnoses for specific patient subgroups long before the overarching system registers a decline in performance metrics (7).

3 Operational Risks of AI in healthcare

Extending the risks presented by AI being inherently non-deterministic, the operational integration of probabilistic models introduces deep sociotechnical and technical challenges. While traditional digital tools generate uniform outputs under fixed parameters, machine learning architectures suffer from a lack of reproducibility; feeding the exact same clinical data or transcript into a generative model multiple times can yield entirely different outputs with varying profiles of error. Therefore there are unique risks associated with AIaMD:

3.1 Taxonomy of AI-Induced Risks in Clinical Environments

3.1.1 Statistical and Distributional Risks (Algorithmic Decay)

- **Data Drift (Covariate Shift)** Data drift (formally known as covariate shift) occurs when the marginal distribution of the input data changes over time, but the conditional distribution of labels given the input remains the same. In practice, this means the relationship between the inputs and outputs remains static, but the model is forced to process operational data outside its training distribution (6). Data drift acts as a "soft failure"

mechanism, causing a gradual, decline in performance. For example, if a hospital upgrades its radiology department from an old MRI machine to a higher resolution MRI machine, the subtle changes in pixel intensity, contrast, and noise ratios alter the input data. A diagnostic AI trained entirely on the old images may misinterpret these hardware artifacts as biomarkers of issues. This can cause false-positive alerts that disrupt clinical workflows, directly impacting patient management, and key medical decisions. AIaMD systems are acutely sensitive to this type of cross-environment (spatial) and within-environment (temporal) variation. As highlighted in Guan et al.(7) , a simple routine software or hardware update to a scanner can drastically degrade an AIaMD's performance. Consequently, strict Post-Market Surveillance (PMS) protocols require organizations to run integrated monitoring dashboards to track data input quality and execute version specific recalibration whenever there is infrastructural hardware changes.

- **Concept Drift (Model Drift)** Concept drift occurs when the conditional distribution of labels given the inputs change; meaning the actual real world relationship between the input features and the target clinical outcome shifts, rendering the model's internal logic obsolete. Evolving clinical protocols, new medical guidelines, or the emergence of new diseases can trigger concept drift. For example, if an infectious disease triage model uses historical respiratory symptoms to predict mortality, the emergence of a highly contagious but significantly less lethal viral variant fundamentally changes that historical relationship. The AI suffers from "knowledge staleness" and continues to apply outdated diagnostic criteria, flagging thousands of low-risk patients for intensive care, exhausting limited hospital resources. (7) Concept drift is a primary driver of model degradation. Because the clinical reality changes while the model knowledge remains static, AIaMD tools will output perfectly formatted but clinically incorrect predictions. To combat this, regulatory frameworks recommend adaptive update strategies, continuous learning loops, and real-time monitoring systems (like the FAMOS dashboard) that actively track AI inference values over time to catch performance decay before patient harm occurs. (2)

- **Edge Case and Out-of-Distribution (OOD) Fragility**

Edge case and out-of-distribution (OOD) fragility represents a vulnerability where medical AI systems experience failure when encountering rare clinical presentations or data patterns completely absent from their training data. This fragility stems from a fundamental technical mechanism; standard neural networks lack an innate capacity to quantify their own uncertainty (18) , causing them to generate incorrect outputs when exposed to inputs that differ substantially from their foundational development data. For instance, an automated dermatology AI evaluating a skin lesion on a patient with a rare genetic disorder like epidermolysis bullosa may fail to comprehend the broader clinical context, misclassifying a highly aggressive melanoma as a standard benign blister typical of the pre-existing condition. In real-world operational environments, these OOD anomalies frequently emerge from subtle demographic shifts, changes in hardware acquisition settings, or rare pathologies. Without runtime anomaly detection or passive monitoring frameworks to flag these silent distribution errors, overworked clinicians vulnerable to automation bias can easily accept these flawed diagnostic conclusions, allowing them to propagate through the clinical workflow and result in severe downstream patient harm.

3.1.2 Socio-Technical and Cognitive Risks (Human-AI Interaction)

Automation Bias: A socio-technical risk deeply rooted in human cognition's tendency to seek the path of least resistance. It manifests as an active psychological heuristic where a clinician implicitly trusts an automated system's output, overriding their own clinical judgment, or immediate physical senses (19) . When AIaMD is generally reliable, over time clinicians develop

more laxity in oversight of the output (20), which in some cases if not double checked can cause detrimental downstream effects. If an AI delivers a flawed narrative, miscalculates risk or suggest incorrect treatment plan, a time pressured fatigued clinician may blindly sign off on the output, permanently writing it into the patients EHR, creating a ripple effect of bad data for future care teams. In real world hospital deployments, real time post market surveillance dashboards have explicitly captured this behavior. For instance, a continuous multi-site evaluation of chest X ray algorithms highlighted in the MHRA Airlock (2) proved that clinician agreement with AI recommendations shifted drastically depending on operational context. Clinicians working irregular hours or facing severe time pressures exhibited a statistically significant susceptibility to automation bias. This underscores that AIaMD cannot be evaluated purely as an isolated algorithm; its safety is fundamentally tied to the sociotechnical dynamics of the human/AI team.

Automation Omission Errors This error subsets from Automation Bias, specifying a clinician's failure to notice an acute clinical anomaly or systemic patient deterioration because the AI system missed to issue an alert or document the finding. Cognitive testing reveals a stark asymmetry in how clinicians process machine errors: commissions (active bad advice) are significantly easier to spot and reject than omissions (20). Catching an omission requires a clinician to recall missing conversational elements or unstructured visual details from hours prior. Because clinicians are conditioned to await the automation, the downstream consequence is delay in care. Anomalies like an unflagged crashing vital sign or a missing drug allergy entry can silently propagate through the clinical workflow until the patient suffers severe harm. This risk directly impacts AIaMD text-summarization tools, Large Language Models (LLMs), and triage software. In benchmarking evaluations featured in (10), omission rates represent a critical "hard failure" mode across frontier language models. For instance, an algorithm may be fine-tuned to be aggressively risk-averse to prevent active hallucinations, but this exact optimization often causes a secondary spike in omission errors. When an AIaMD leaves out subtle flag of symptoms or vital diagnostic parameters due to a corrupted data stream, a narrow context window, or greedy decoding rules, it provides a form of "unsafe reassurance" that can blind the human in the loop (20).

Automation Commission Errors Another subset of automation bias is commission cognitive errors, unlike omission cognitive errors, it encompass when the medical professionals actively accepts and executes incorrect or fabricated recommendation created by the automation output (20). The primary driver of commission errors is the fluent presentation of AI text and metrics, which can easily mask underlying logical flaws. Critical downstream issues include ordering unneeded invasive biopsies, or executing incorrect surgical plans based purely on machine generated fabrications. Therefore distorting the reality of the clinical presentation and overriding the traditional redundant safety checks established within clinical teams. For generative and predictive AIaMD, commission errors are heavily tied to model hallucinations and poor calibration. As explored across multiple clinical case studies in the MHRA Airlock and Van Oyen et al. (2) (10), models frequently output incorrect diagnostic classifications with extreme verbal and statistical confidence. Furthermore, when an AIaMD operates as a "black box" without traceable explanations, clinicians lack the ability to interrogate the algorithm's intermediate logic. Without layered explainability tools (such as LIME or SHAP feature attribution metrics) to show why an AI made a possible "wild" recommendation, clinicians cannot distinguish a true clinical emergency from an artifact or data drift event, directly inducing fatal clinical actions.

De-skilling (Cognitive Atrophy) De skilling is a long term effect of automation bias, where the long term over-reliance on automation causes the eventual loss of core human diagnostic, technical or procedural capabilities. With AI introducing itself into the medical field, it plays as a relief of stress for established medical professionals, however for the incoming class of clinicians who start to learn with AI, they have no established memory before AI. When automated tools routinely manage data collection, triage, and primitive diagnostic loops, junior clinicians, residents, and nursing staff can fail to develop the intuitive, pattern recognition skills

compared to the established professionals who have forged knowledge from manual reasoning. A future dilemma can be a medical workforce that is highly fragile; if the automated framework experiences a critical server failure, an IT outage, or a localized network disconnect, the clinical team's manual diagnostic accuracy can severely plummet due to the automation bias present from the early stages, jeopardizing patient safety. (2) (10) (21) . As AIaMD evolves from a reactive diagnostic aid into a proactive co-pilot integrated into clinical workflows, defining the boundary of human/machine boundaries becomes paramount. Medical literature warns that when a device acts as a continuous "gatekeeper" (mandating human validation for every step), it creates severe cognitive bottlenecks and accelerates deskilling, as human input is reduced to a repetitive, automated "click-through" verification loop. To preserve clinical capability, AIaMD deployment strategies must utilize explicit risk adapted gating, ensuring that ambiguous cases are routed to enforce active human problem solving, rather than allowing automation to flatten the clinical reasoning (21) (10) .

3.1.3 Generative and LLM-Specific Vulnerabilities

Medical Hallucinations (Confabulation) Hallucinations refer to the generation of content from an unidentifiable or non-existent source, with the content submitting fabricated or incorrect facts as factual to the user. (7) . These distortions directly corrupt the clinical narrative. Because generative models pre-compute probabilities for the next most likely word rather than checking factual ground truth, they can confidently invent medical histories, drug dosages, or diagnostic tests. This could leave patient record filled with invisible, unsafe falsehoods that can mislead future care teams, alter diagnoses, and cause severe patient harm, if unchecked. When LLMs are utilized as an AIaMD for automated clinical documentation or summarization, managing hallucinations is a primary regulatory hurdle. Real world validation testing inside the (2) framework showed that baseline models produced substantial factuality and faithfulness to hallucinations. To mitigate this device risk, manufacturers are developing architecture overrides like Retrieval Augmented Generation (RAG) to explicitly ground clinical outputs in verified sources (such as NICE guidelines).

Context Window Degradation + Omissions As the technical precursor to cognitive omissions risk the omission risk from the AI sits as a the tendency of LLMs to drop, ignore, or truncate critical information located in the middle or at the tail end of long contexts. (2) . Patient records, and clinical transcripts are dense, messy, multi-page environments. If an AI system truncates or skips a single sentence hidden deep inside the input data, such as a critical medication allergy or a prior surgical complication, the downstream effect is an incomplete medical profile. Clinicians relying on the summary will make high stakes diagnostic or surgical decisions completely blind to unexpressed patient risks, risking severe possible operative complications. Omission errors represent a severe "hard fail" category for text-to-text generative medical devices. Data from Oukelmoun et al. (8) underscores that automated text evaluators struggle immensely with identifying missing information. Furthermore, as shown in Van Oyen et al. (10) , when safety focused prompting or greedy decoding is applied to an AIaMD to aggressively suppress active hallucinations, the model's omission rate frequently increases as a trade-off. It requires secondary validation layers (such as EmbedKDECheck) running independently to ensure semantic integrity across raw inputs and summarized device outputs.

3.1.4 Architectural + Systemic Blind Spots

Black-Box Opacity (Lack of Explainability) The Black Box represents the inability of developers, regulators and clinicians to audit, trace, or comprehend the underlying logic of an AI output/ the weight distributions that a deep neural network utilized for a specific prediction (2) . Black-box opacity introduces severe clinical decision dilemmas. If an algorithm recommends against a guideline proven therapy or flag an unidentifiable risk profile, the physician cannot

determine whether the device has accurately identified a biomarker or has suffered from systemic data drift. This erodes clinical trust, creating massive liability and safety concerns, trapping the provider between blindly trusting an opaque device or rejecting a potentially life-saving prediction. AIaMD software adapts to dynamic data and changes in clinical protocols over time, explainability serves as a core mechanism for oversight and clinical integration. As detailed in MHRA Airlock (2), the OncoFlow case study, developers are forced to build explainability frameworks to meet regulatory scrutiny. These require the device to actively log its rationales, implementing quantitative feature importance testing (such as SHAP or LIME metrics), and provide layered, context-aware visual explanations to the end user to assure safe teaming.

Demographic Bias + fairness failures (Equity Risk) This risk presents in degradation of model performance, calibration or accuracy across specific demographic subgroups (stratified by race, ethnicity, sex, or socioeconomic class) (22) (7). Consequences of demographic bias would cause amplification of healthcare inequalities. Where an unrepresentative model is deployed in real-world environments, would miscalculates risk or misclassifies clinical conditions for minoritized patient groups. This causes a discriminatory denial of preventative interventions and highly inequitable care allocations. Medical AI models are highly sensitive to cross-environment and spatial variation. When a device trained in an environment with specific demographics is deployed in a distinct locale without retraining, its diagnostic capabilities degrade heavily. Evidence from Guan et al. (7) highlights that performance metrics alone cannot guarantee safety; monitoring tools must actively run stratified fairness audits and out-of-distribution (OOD) tests across demographic spaces to block hidden technical debt and safeguard technical accountability.

algorithmic Feedback Loops (Self-Fulfilling Prophecies) If situated with the over reliance of AI to a point its prediction heavily drives human clinical action, and that action subsequently alters the clinical course in a way that falsely validates the AI's original biased prediction; is the definition of an algorithmic feedback loop. Where the previous fabricated output influences the outputs after. (7)

Risks of a feedback loop is an automation which, self-reinforces its inflation of accuracy metrics that is hiding underlying bias. If a model severely overestimates mortality risk, and the clinical team responds by withdrawing aggressive treatment, the resulting patient death is logged by the hospital database as a successful, accurate prediction. This creates an unmonitored baseline error that permanently stains the clinical dataset, systematically cementing bad guidance as standard practice. The feedback loop represents a dangerous threat to AIaMD in long term reliability. Longitudinal simulation data from Guan et al. (7) demonstrates that repeated internal updates on historic, looped EHR data cause models to gradually degrade over time, dramatically increasing false-positive rates even while standard diagnostic metrics appear perfectly stable. To protect the pipeline from structural corruption, AIaMD protocols require adaptive monitoring that isolates active clinician model interaction logs rather than simply trusting long-term performance statistics.

4 Current Evaluation Paradigms and Reporting Gaps

4.1 Pre-Deployment Testing and Benchmarking Limitations

Before new AIaMD technologies enter the market, developers perform rigorous pre-deployment testing using historical and uniformly augmented data to simulate real world challenges (7). The primary benchmark metrics used to evaluate these devices include the Area Under the Receiver Operating Characteristic curve (AROC), accuracy, precision, and recall. While these metrics effectively determine the mathematical capability of the AI, the benchmark data rarely reflects the full complexity of live clinical environments. Consequently, once deployed in the real world, AIaMD remains highly susceptible to performance degradation caused by model and data drift. Research by the U.S. Food and Drug Administration (FDA) (23) (24), indicates

that continual learning is vital to keeping AI software viable throughout its market lifecycle. However, developers must carefully evaluate the new data used to update these models to prevent algorithmic feedback loops and avoid compounding pre-existing biases.

Empirical evidence highlighting the limitations of relying solely on historical training data is documented in (25). Using a clinical order recommender system as a test environment, researchers found that models predicting future (2013) orders performed significantly better when trained on a single month of recent data from 2012 rather than 12 months of older data from 2009. The recent-data model achieved a higher ROC AUC (0.91 vs. 0.88), better precision (27% vs. 22%), and superior recall (52% vs. 43%, all). Notably, the AI still outperformed human-authored recommendation sets, which scored a 0.81 ROC AUC, 16% precision, and 35% recall (25). This clearly underscores the practical necessity of continual learning for AIaMD. Supporting this, case studies like (26) evaluate how machine learning models degrade over time when predicting hospital mortality rates. Although these models remain generally effective one year post-deployment, they still yield lower performance scores compared to their initial deployment state. Furthermore, Young et al. (26) exposes the limitations of using standard 10-fold cross-validation to predict long-term reliability, emphasizing instead the need for adaptive monitoring to maintain clinical robustness (7).

If left unnoticed, these performance decays can cause severe consequences in a medical environment. To address these vulnerabilities, MHRA Airlock (2) explored the AutoMedica Smart-Guideline tool to mitigate the risks of LLM hallucination and non-determinism. SmartGuideline, an emerging tool trained on the most up to date medical data, was benchmarked against ChatGPT-4. The testing revealed that SmartGuideline successfully cut hallucinations down to 0% compared to ChatGPT-4's 7.5%. However, this came at the cost of an 8.5% omission rate, whereas ChatGPT-4 maintained 0% omissions. While these scores may initially fall within acceptable clinical metrics, failing to update the models over time will inevitably reduce their reliability, as demonstrated by (25) (26). Relying on healthcare workers to catch these subtle software degradations between scheduled updates is a heavy burden; data from (13) reveals that less than half of surveyed medical professionals find time to complete AI training, and even fewer feel competent enough to consistently identify systemic AI issues.

In light of these challenges, real-time post-deployment monitoring is increasingly viewed as the most pressing priority to guarantee the long-term reliability and robustness of clinical AI (7).

4.2 Post-Market Surveillance and the Yellow Card Framework

In the UK, three main bodies regulate the medical field: the Medicines and Healthcare products Regulatory Agency (MHRA), the Care Quality Commission (CQC), and the National Institute for Health and Care Excellence (NICE) (13). While the CQC evaluates the operational competence of healthcare facilities, NICE establishes national clinical guidelines and treatment funding approvals. Meanwhile, the MHRA governs the usage and deployment safety standards for medical devices. Together, these bodies share the responsibility for tracking long-term safety, relying on PMS frameworks. Traditionally, the MHRA maintains this surveillance through retroactive vigilance pathways, utilizing the Manufacturer's Online Reporting Environment (MORE) for corporate adverse incidents and the Yellow Card Scheme for clinical end users and patients. These reporting schemes add utility to PMS by establishing structured data collection paths to capture adverse events and providing a mechanism to oversee subsequent corrective actions. However, with the rise of AIaMD, these retroactive reporting frameworks are proving insufficient. Because legacy reporting channels were fundamentally designed for static medical products, they fail to dynamically capture non-linear AI failure modes, such as silent model drift, automation bias, and clinical hallucinations, before patient harm has already occurred.

4.2.1 The Yellow Card Scheme

The Yellow Card Scheme applied by MHRA is the UK's historical, national vigilance framework managed by MHRA to capture adverse incidents and medical product deficiencies. Originally established in the 1960s in the wake of the Thalidomide tragedy to monitor unforeseen pharmaceutical side effects, its regulatory scope has since expanded to encompass conventional medicines, alternative delivery systems like e-cigarettes, and physical medical devices (27) (28) . In the current AI landscape, however, the scheme exhibits a structural and architectural mismatch. Because it relies entirely on a voluntary, human driven reporting workflow where a clinician or patient must manually identify an error after an event has occurred, it is incapable of catching the "silent failures" or sub-visual semantic drift attributed to probabilistic AI models. Quantifiably, while the MHRA assesses over 101,000 adverse drug reaction reports and roughly 56,000 medical device incidents annually, an microscopic fraction represents software or AI anomalies due to this massive manual reporting overhead (11) . The Yellow Card Scheme is differentiated from modern frameworks by its retrospective, user dependent, and reactive nature, serving as a lagging safety net rather than an active operational guardrail.

4.2.2 Manufacturer's Online Reporting Environment (MORE)

The Manufacturer's Online Reporting Environment (MORE), owned by MHRA, is a statutory, portal based compliance pathway mandated by the MHRA under the UK Medical Devices Regulations (UK MDR 2002) for the direct submission of post market surveillance data by medical device manufacturers. Initially designed to formalize and streamline vigilance workflows for traditional hardware medical technologies, MORE provides a protocol for commercial vendors to log serious adverse incidents, periodic summary reports (PSRs), and macro level trend analyses (5) . While MORE remains a mandatory legal mechanism within the AIaMD landscape, its clinical effectiveness is bottlenecked by its vendor dependent and retrospective architecture, similar to the yellow card scheme. Manufacturers are bound by strict statutory timelines, varying from 2 to 15 days based on incident severity, to investigate and report failures, which requires them to first become aware of the glitch through external hospital complaints or localized debugging (5) . This framework cannot dynamically audit non-deterministic algorithms or catch localized model calibration decay as it unfolds on site. Unlike clinician facing systems, MORE is strictly a B2B regulatory compliance channel, yet it shares the same structural limitation of being a historical archive rather than a proactive of live runtime errors.

4.2.3 Routine Clinical Auditing

Routine Clinical Auditing represents a localized, department level quality assurance framework where in clinical and informatics teams periodically review deployed technologies against a validated clinical reference standard considered the "ground truth." Conceived as an operational tool for clinical governance and professional peer review, auditing within this AI landscape acts as a manual mechanism to calculate human/AI disagreement rates, track diagnostic accuracy, and reveal demographic performance variations. although over 90% of NHS trusts maintain the baseline administrative infrastructure via designated Medical Device Safety Officers (MDSOs) to coordinate these safety reviews, the methodology remains heavily resource-constrained and intermittent (29) . While highly valuable for spotting scanner specific anomalies or localized population shifts, clinical auditing operates purely as a retrospective snapshot in time. It requires substantial clinical hours to manually re evaluate historical AI outputs against final pathology or long-term patient outcomes, meaning that dangerous algorithmic hallucinations or omissions can pollute the electronic health record (EHR) for weeks or months before a trend is uncovered. It is praised for its high clinical specificity and localized oversight, but lacks the automation and immediacy required to protect daily, quick clinical workflows. This is reviewed by CQC.

4.2.4 Continuous Real Time Monitoring

Continuous Real Time Monitoring represents the modern, software native paradigm designed explicitly to address the probabilistic vulnerabilities of non-deterministic clinical AI. Unlike legacy systems, this framework introduces an automated, vendor neutral middleware layer directly into the active clinical data pipeline to continuously evaluate model inputs and outputs in real time. Within the UK healthcare ecosystem examples are; national initiatives like the Federated AI Monitoring Service (FAMOS), spearheaded by health tech enterprises such as Newton's Tree and supported by Innovate UK, which has been successfully piloted at major acute centers including the Leeds Teaching Hospitals NHS Trust (30). Architecturally, this methodology is designed specifically for the AI landscape: it monitors the dynamic interaction between raw patient data (such as ambient consultation audio) and the model's generated summaries to flag statistical divergence, model drift, and safety-critical omissions at the exact point of care. Continuous real time monitoring is fundamentally differentiated from all preceding systems by being proactive rather than reactive; it acts as an automated, immediate algorithmic interceptor that alerts the clinician and updates safety dashboards before a flawed output can propagate into the legally binding EHR. This is not enforced but is reviewed by the MHRA.

5 Modernizing Surveillance: Integration and Analytics

Historical reliance on manual incident logging, embodied via the UK's Yellow Card Scheme, is misaligned with the rapid, probabilistic nature of AIaMD. The Yellow Card framework operates as a reactive, retrospective safety net that requires a clinicians or patients to manually identify and report an error only after an adverse event has already occurred. However, in a pressured environment like healthcare, professionals can lack the operational bandwidth and the technical visibility to manually detect the silent failures in AIaMD or subtle semantic drift of probabilistic AI models. As machine learning architectures continuously adapt and can output flawed predictions with high confidence, waiting for a human to notice a data drift anomaly or a clinical hallucination creates a dangerous safety vacuum between manufacturers and clinical reality. The traditional method of retroactive reporting is incapable of dynamically auditing non-deterministic algorithms as they are fully dependent on the human in the loop, who can be susceptible to automation bias.

To help combat the architectural vulnerabilities of using AIaMD, this paper proposes the integration of an automated, passive monitoring framework, an algorithmic interceptor that sits within the clinical data pipeline, between the raw clinical input and the AI's generated output. This middleware layer acts as a real time validation engine, actively scanning to help prevent the three primary AI induced risks: clinical hallucinations, safety-critical omissions, and black box, unexplainable logic. Instead of relying on manual oversight, the algorithm automatically highlights these critical areas of concern directly within the user interface, immediately drawing the clinician's attention to where verification is needed. Furthermore, the system performs targeted deterministic checks against basic clinical risks, such as actively flagging the transcription of high risk drugs to ensure the AI's output aligns with established safety protocols. By proactively surfacing these potential errors, the framework forces the clinician to review specific, high-stakes variables. This active engagement breaks automation bias and mitigates long-term clinical deskilling (19). It ensures the human remains actively engaged in the clinical reasoning loop, intercepting dangerous anomalies before they can permanently propagate into the EHR, while reaping the benefits of AIaMD.

The advantage of this middleware methodology is that it sits on the clinical pipeline as well keeping its digital retention. By operating as a middleware layer, the system shifts the AI evaluation from a lagging, retrospective task to a proactive, on-the-go intervention. Furthermore, as it is a native software layer, this system can capture and store an auditable history of flagged anomalies, algorithmic drift, and AI degradations to be reviewed later. This digital retention

is critical because expecting a clinician to recall an omitted conversational detail from hours prior accurately relies entirely on fallible human memory, a major point of cognitive failure in high-stress environments (31). By maintaining a ledger of these errors, the middleware does not replace the Yellow Card scheme; it empowers the use of it. The middleware aids the reactive reporting framework as anomalies are held by the software rather than the clinician's brain. Ensuring that historical data is preserved, allowing for detailed, evidence-based reporting to regulatory bodies while completely removing the manual logging burden from the clinical staff. This potentially could help close the gap between manufacturers and the people that they help.

6 The Proposed Solution: Targeted Evaluation via Ambient AI Scribes

Within the United Kingdom's AIaMD ecosystem, AVT has emerged as a prominent driver of operational efficiency and rapid return on investment (ROI) (32). The NHS England Ambient Voice Technology Supplier Registry formalizes this landscape, validating tools for clinical deployment (33). Platforms like TORTUS have achieved widespread usage with over 1,000 hospital integrations (34), alongside Heidi Health, which has experienced extensive grassroots adoption across primary care general practices (GPs) and health clinics (35). Short term implementation studies demonstrate significant reductions in administrative overhead. Independent validation metrics compiled by KLAS Research evaluating SUKi a major clinical intelligence platform recorded a 26.8% reduction in note creation time at McLeod Health and a 21% reduction at FMOL Health, alongside an 18.3% to 22% organic increase in monthly patient encounters driven by recovered provider capacity (32). Furthermore, early enterprise deployment of these tools correlates with measurable declines in clinician burnout, dropping from 51% to 38.8% within six months of usage (32). Domestically, pilot implementations of TORTUS within NHS emergency settings have achieved a 13.4% increase in Accident & Emergency (A&E) productivity while returning 23.5% more direct, face-to-face care time to patients. (34)

6.1 AVT Embedded Risk Mitigation

To navigate clinical safety requirements, AVT tools incorporate active risk aversion mechanics alongside a mandatory "human-in-the-loop" verification framework. For example, TORTUS advertises a 92% hallucination mitigation rate via localized alignment tuning, while Heidi Health details structural verification protocols under the NHS DCB0160 clinical safety standard to audit structural note formatting, speaker attribution, and clinical omissions before data is committed to the patient record. Despite these safety assertions, the legal and operational accountability for verifying text fidelity remains strictly on the healthcare professional (34) (35).

However, relying entirely on human verification at the point of care introduces deep sociotechnical and cognitive vulnerabilities (5). Frontline medical staff frequently operate under severe time constraints and heavy administrative workloads, rendering them highly susceptible to automation bias (20) (13). Overextended clinicians easily develop a psychological heuristic that accepts fluent, structured machine-generated notes without sufficient critical scrutiny, leading to the silent propagation of errors into the Electronic Health Record (EHR) (13) (7). While active commission errors (fabricated facts) are periodically caught during review, catching automation omission errors requires a clinician to manually cross reference the summary against complex conversational details from prior hours, relying heavily on fallible human memory (8). Furthermore, because these systems are fundamentally probabilistic rather than deterministic, they are highly prone to long term performance degradation (7). Real world deployments inevitably suffer from data drift (covariate shift) as clinical demographics or operational recording environments evolve (6), alongside concept drift as internal performance calibrations decay over time (7). Crucially, standard safety mechanisms like the UK's historical Yellow Card scheme

function purely as retrospective, lagging vigilance pathways (28) . Because the Yellow Card framework depends entirely on manual event logging after an adverse clinical incident has occurred, it creates a severe regulatory vacuum (10) . It is structurally incapable of capturing live algorithmic decay or sub-visual semantic failures before they translate into direct patient harm (7) .

6.2 AVT System Architecture

Architecturally, modern AVT applications do not deploy as standalone operating systems; instead, they function as an application layer integrated directly into existing clinical software frameworks. Clinicians typically interact with these tools via mobile operating systems (iOS and Android) running secure application layers at the edge, or through native desktop environments (Windows and macOS) hosted on enterprise hospital hardware (35) . The processed textual output such as a structured Subjective, Objective, Assessment, and Plan (SOAP) note is delivered directly to the user interface via an Application Programming Interface (API) layer mapping to major enterprise EHR platforms like Epic, Oracle Health, or primary care architectures like EMIS (36) .

Because these applications require direct clinician interaction on the screen prior to charting execution, an automated, vendor-agnostic safety net can be successfully introduced. Rather than seeking server side vendor API access, a lightweight browser extension or desktop overlay can sit directly within the client side pipeline. By reading the Document Object Model (DOM) or intercepting the screen text at the point of review, this middleware layer can dynamically parse the text payload, execute real time diagnostic cross checks, and flag safety critical anomalies before they permanently write into EHR.

6.3 What this work adds to clinicians

The operational reality of deploying adaptive documentation tools within NHS trusts can be codified. Where we can analyse where the highest risks are likely to occur and highlight it to the human to check. This work is not to try full eliminate issues that occur from non-determinism, but to use deterministic algorithms to help mitigate known issues that can cause the highest risk. Examples of high likelihood against high risk issues include the following in :

Table 1.

Table 1: Hazard Analysis and Risk Mitigation Matrix adopted from GP concerns

ID	Condensed Hazard	Downstream Risk Analysis	Human / Process Control	Deterministic Algorithmic Control
H1	Omission of Clinical Data (Symptoms, History, Exam, Red-Flags)	Misdiagnosis & Incomplete Record: Omitted symptoms, red flags, or exam traits lead to biased care decisions.	Narrate exams aloud. Cross-check summary against encounter; manually insert missing findings/red-flags before saving.	Quality & Red-Flag Checker: Real-time Audio SNR analyzer; rule-based entity extractor comparing transcript against summary output.
				Continued on next page

Hazard Analysis and Risk Mitigation Matrix adopted from GP concerns (continued)

ID	Condensed Hazard	Downstream Risk Analysis	Human / Process Control	Deterministic Algorithmic Control
H2	Data Distortion & Fabrication (<i>Hallucinations, Mis-attribution</i>)	Later Clinical Errors: Unsaid facts or misattributed speaker statements trigger unwarranted testing and corrupt history.	Review note against encounter; strike out ungrounded assertions. Explicitly label clinician hypotheses under Assessment.	Grounding & Attribution Rules: Substring/entity overlap filters to highlight ungrounded terms; hard speaker diarization mapping rules.
H3	Format & Granularity Non-Compliance (<i>Verbosity, Structure</i>)	Info Overload & Delayed Care: Unstructured, overly brief, or bloated text increases cognitive load and obscures facts.	Apply standard SOAP templates. Trim transcript bloat into concise SOAP entries; regenerate if depth is inadequate.	Schema & Density guardrails: Token/entity density threshold rules; hard length ceiling limits; strict JSON/SOAP schema validation.
H4	EHR Placement & Identity Errors (<i>Wrong Field, Wrong Chart</i>)	Severe Legal & IG Breach: Documenting on wrong chart/field causes misdiagnosis, improper prescribing, and data breaches.	Perform two-identifier checks (Banner + DOB/NHS No.). Double-check target EHR field headers before posting/pasting.	Session & Context Locking: System-level hook matching active EHR patient ID against session ID; context-aware field API locking.
H5	Governance & Consent Failure (<i>Missing Consent, Complaints</i>)	Legal & Regulatory Breach: Operating without consent damages trust, violates IG standards, and risks regulatory sanction.	Explain tool upfront, obtain verbal consent, and document choice. Pause tool immediately if patient opts out.	Consent Boolean Lock: Hard UI rule requiring a validated <code>Consentobtained == True</code> flag before audio pipeline buffer initiates.

To alleviate this structural vulnerability, the proposed passive monitoring framework automates the validation layer within the textual data pipeline. By operating as an independent middleware layer, the system executes real time, deterministic text checks to instantly parse and flag Look alike, Sound alike (LASA) pharmaceutical errors, structural formatting deviations (H2), or missing objective examination parameters (H1) before the note can be committed to the record. Furthermore, by tracking background metadata such as edit distance variances and clinician review time, the system can algorithmically identify unverified note submissions that indicate the human safety loop has degraded. This targeted automation strips the primary logging and filtering burden away from the clinician, providing a real time safety scaffold that actively surfaces probabilistic failures before they can propagate into EHR.

7 This Paper's Objectives

The core objectives of this research are structured across three distinct domains: algorithmic evaluation, system engineering, and regulatory policy.

algorithmic Evaluation: The primary objective is to evaluate different deterministic computational methods for detecting clinical hallucinations, safety-critical omissions, and major operational risks within AVT-generated SOAP notes. A deterministic approach was deliberately chosen to ensure the monitoring framework remains strictly auditable and reproducible, mirroring the regulatory stability of traditional SaMD. While the empirical evaluation of these methods ultimately yielded negative results regarding their immediate, out-of-the-box viability, primarily exposing a severe register mismatch between spoken

clinical dialogue and structured written summaries, the tests provide vital architectural insights and demonstrate clear promise for future algorithmic refinement.

System Engineering (Proof of Concept): To operationalize these insights, a secondary objective is to build a functional delivery mechanism in the form of a client-side browser extension and local API. This Proof of Concept (POC) aims to actively highlight potential textual errors to the clinician to mitigate automation bias, while passively recording edit statistics over time to monitor longitudinal model drift. Designed as a modular infrastructure, this POC is capable of seamlessly integrating any of the proposed deterministic modules once they achieve validated clinical efficacy.

Regulatory Policy Recommendation: Finally, building upon the technical and empirical findings, this work aims to synthesize the research into a formal white paper policy proposal. This policy will outline actionable guidelines for healthcare regulators, such as the MHRA and NHS Trusts, regarding the safe deployment and continuous post-market surveillance of AIaMD. By formally defining the necessity of real-time, deterministic middleware, this white paper seeks to address the critical safety vacuums left by retrospective reporting frameworks like the Yellow Card scheme, establishing a modernized standard for clinical AI governance.

8 Test data

To benchmark and validate deterministic safety algorithms for ambient clinical documentation tools, a combination of conversational clinical corpora, medical knowledge bases, and annotated biomedical entities is required. This section details the test data being used. The main test set is the PriMock57 dataset with coded augmentations.

8.1 PriMock57 Corpus (babylonhealth/PriMock57)

Overview & Description

Released by Babylon Health, **PriMock57** consists of 57 mocked primary care consultations conducted by licensed clinicians and role playing actors. The dataset reflects real comprises 57 mock (GP) consultations and contains fully diarized transcripts paired with GP ground truth consultation notes (37) .

File Structure & Organization

```
PriMock57/
|-- data/
|   |-- transcripts/          # Speaker-diarized transcripts
|   |-- notes/               # Clinician-written consultation notes (.json / .txt)
|   |-- Audio/               # the raw audio files
|   '-- metadata.json        # Encounter metadata (chief complaint, duration)
'-- README.md
```

Contents & Annotations

- **Diarized Transcripts:** Multi-turn dialogue with explicit speaker role tags (Doctor:, Patient:).
- **Audio:** Audio files split into doctor and patient, the timestamps present in the transcripts
- **Structured GP Notes:** Clinical entries covering *History of Present Illness (HPI)*, *Past Medical History (PMH)*, *Physical Examination*, *Assessment*, and *Management Plan*.

Dataset Statistics & Distribution

- **Encounters:** 57 primary care consultations.
- **Domain:** UK Primary Care / General Practice (cardiovascular, respiratory, gastrointestinal, and musculoskeletal cases).
- **Annotations:** Speaker diarization labels and structured notes.

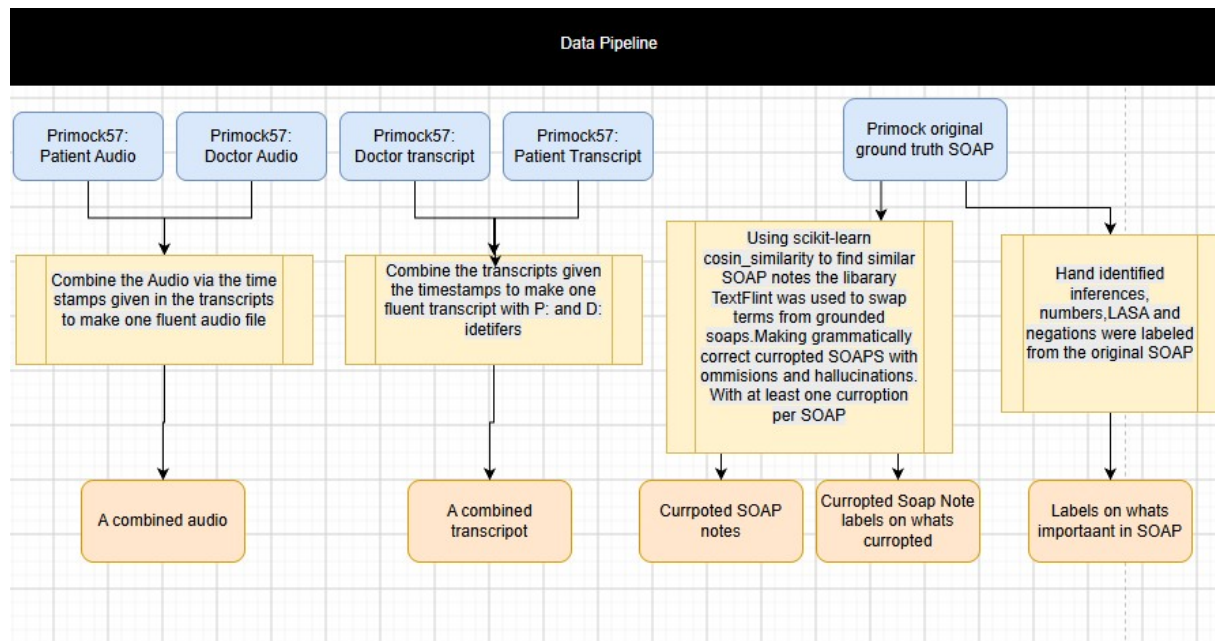


Figure 4: Making the Data

Safety System Application (Omission, LASA, Hallucination)

- **Speaker Mis-attribution Detection:** Benchmarks whether safety guardrails detect clinician hypotheses (e.g., "We must rule out angina") incorrectly assigned to the patient ("Patient reports angina").
- **Primary Care Omission Auditing:** Ensures primary care red flags (e.g., unexplained weight loss, acute chest discomfort) mentioned in consultation audio appear in the plan.

Target Hazard Mapping Table 1

- **H1:** Missing clinical red flags in primary care. This dataset includes simulated primary care from professionals and actors.
- **H2:** Mis-attribution of speaker statements (Patient vs. Clinician). This dataset includes both audio and a written transcript.
- **H3:** Managing note granularity and text length. The dataset includes ground truth SOAP notes in SOAP format.

8.1.1 How the Dataset Was Augmented

Beyond the raw PriMock57 (37), three data preparation pipelines were built to adapt the corpus to the specific evaluation needs of the safety system: (i) a combined-audio track suitable for a single stream ambient scribe pipeline. (ii) a library driven corruption pipeline that injects deterministic, non LLM *bad* labels into the gold notes for evaluating the safety checkers. (iii) an extraction of structured ground truth labels from the original gold SOAP notes for evaluating the note extractor.

Combined Doctor/Patient Audio

PriMock57 ships doctor and patient audio as separate mono WAV files per consultation. Since the target ambient scribe pipeline consumes a single audio stream, this project combines file by consultation ID and combines them into a single two channel stereo WAV (doctor on the left channel, patient on the right) using `pydub's AudioSegment.from_mono_audiosegments`. Because the two tracks are already time synchronized recordings of the same consultation, no timestamp alignment is required; the only correction applied is a tail end sample count match (truncating both channels to the shorter track's exact sample count, a discrepancy of at most ~ 960 samples / ~ 60 ms observed across the corpus) before the channels

are combined. The result is exported at the source WAV’s native sample rate and bit depth. A related but separate step, this project also performs a timestamp aligned merge at the text level: it interleaves the doctor and patient TextGrid transcripts by sorting each speaker’s turns on their `xmin` start time into a single chronological, role-tagged (`d:` / `p:`) dialogue. to simulate a transcript received by an ambient AI This text combination would be used as the input file for the system.

Library Based Bad-Note Generation

Prim57 dataset provides ground truth notes with no corruptions (37) . To benchmark the safety checkers against corruptions that are not themselves produced by an LLM (and therefore do not risk correlated failure modes with an LLM based checker), a deterministic corruption pipeline was implemented.

The library-based pipeline combines:

- **scikit-learn** (`TfidfVectorizer` + `cosine_similarity`) to find, for each gold note, its one to three most similar notes in the corpus by TF-IDF cosine similarity, forming a donor pool for hallucinated content;
- **TextFlint** (`SwapNamedEnt`) to run named-entity recognition on a note, decompose entities into PERSON/LOCATION/ORGANIZATION categories, and swap an entity for a same category entity drawn from a similar note in the donor pool;
- Custom **medspaCy/QuickUMLS** based `MedspacyUmlsChecker`, reused as a truthfulness oracle: before a candidate hallucination is accepted, it is checked against the target consultation’s own transcript (matching UMLS CUI and assertion polarity) so that a coincidentally true statement pulled from a donor note is not mislabelled as a hallucination.

Five corruption actions are drawn from at random (1–5 issues per note, deduplicated by (`type`, `detail_type`, `detail`)): entity swap (with a UMLS drug semantic type check distinguishing a drug name switch from a generic entity swap), whole sentence hallucination spliced from a similar note, sentence omission, a regex based negation flip (`denies` → `reports`, `without` → `with`, stripped `no/not/n’t`, etc.(specific to dataset)), and a numeric augmentation of doses or vitals (a factor of ten shift or a single digit edit). Each injected issue is recorded as a labelled JSON object (`type`, `severity` via a `seperate risk taxonomy algorithm`, `detail_type`, `detail`), giving an exact ground truth against which checker recall and precision are scored. Figure 5 (left) shows the resulting distribution across all 57 notes.

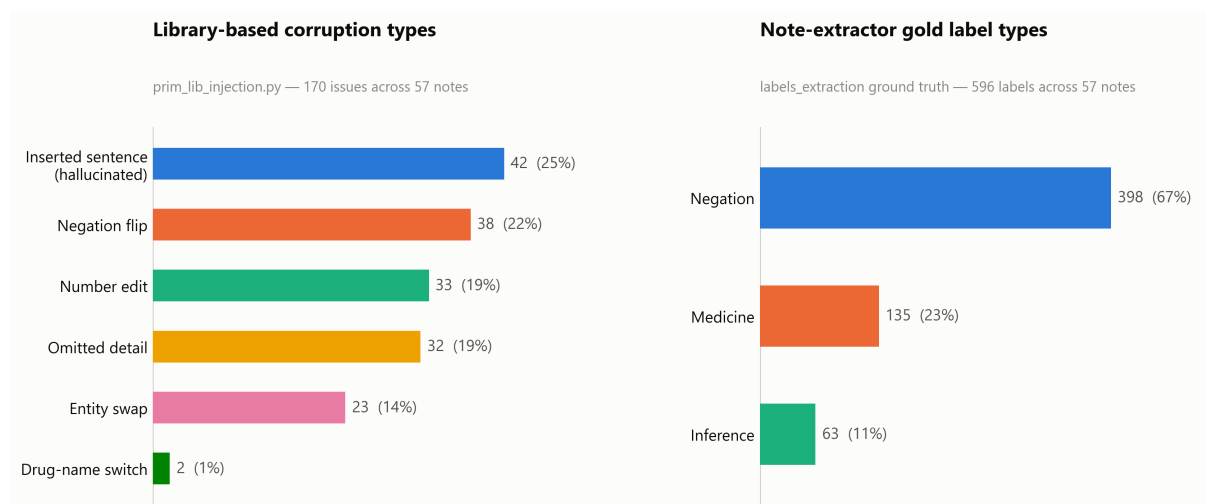


Figure 5: The injected corruption distributions as well as the ground truth type labels.

Identity Labels from the Original SOAP Notes

For evaluating the structured note extractor method, Identity labels were curated independently of the corruption pipelines, ground truth labels were derived directly from PriMock57’s original, uncorrupted GP notes. Within each section, four label categories are extracted per line or clause: **medicine** (UMLS

drug-semantic-type matches via QuickUMLS, filtered by a hand curated denylist to remove coincidental collisions with common words), **negation** (regex cue words such as **no**, **nil**, **nkda**, **denies**, specifically for this dataset), **inference** (hedging/differential language such as **likely**, **r/o**, **?**-prefixed terms, or unconditionally the whole Assessment/Impression section, specifically for this dataset), and **number** (dose, frequency, duration, or vital-sign phrases). The gold labels used for evaluation were curated against the original notes for the medicine, negation, and inference categories; [Figure 5](#) (right) shows their distribution across the 57 notes.

9 Determinism in a non-deterministic space

The backbone of this proposed proactive system is its ability to employ deterministic algorithms that do not suffer from the same extended use downfalls as its probabilistic counterparts [Section 3.1.1](#). The proposed solution is to behave mostly under SaMD, a fixed auditable function that always maps a given input X to Y, against the AIaMD it checks, whose mapping from transcript to SOAP is not fixed nor fully inspectable run to run. Determinism is a core design each method below is measured against as well as its suitability to a constrained hardware environment to simulate working in the medical field. The following section outlines possible deterministic methods that can expand the proactive system. Each method is imposed onto a SOAP checker module to test its viability for checking SOAPS against transcripts, further explained in [Figure 7](#).

While modern machine learning governance often relies on feature attribution frameworks such as SHAP [\(38\)](#) or LIME [\(39\)](#) to combat black-box AI systems, these tools have been excluded from the methods. SHAP and LIME are designed for model explainability, which inputs result in the current output through heavy computational permutations. Running them per transcript, SOAP pair per consultation will prove unfeasible, as the goal of the sentinel system is to run on a standard hospital computer with a relatively quick speed. Furthermore, the aim is not to see why the model made a mistake, but to flag it if it did, so clinicians can quickly correct it. The SHAP and LIME tools can be used by manufacturers when recreating the flag. The four methods selected, medspaCy (rule-based NLP), EmbedKDECheck (statistical density), AlignScore (trained NLI), and SummaC (document-level NLI), were chosen because they can treat the generative AI as a true black box, requiring only the source transcript and the generated output to act as an independent, deterministic adjudicator.

While other evaluation paradigms exist, many using frontier Large Language Models (LLMs) as zero-shot judges, like Bhavik Vachhani et al. LLM as a judge method [\(40\)](#), they were deliberately left unexplored in this empirical testing. LLM evaluators introduce secondary non-deterministic risks, incur prohibitive computational costs, and rely on external cloud APIs that violate the strict data residency and local hardware constraints required for a secure, SaMD aligned hospital deployment. These LLM methods are also often used within the AVT safety checks as well [\(34\)](#).

Other deterministic information extraction tools exist, such as Open Information Extraction (OpenIE) frameworks like MinIE [\(41\)](#); they were deprioritised in favour of a targeted methodological progression. The selected methods were chosen not as isolated tests, but as a continuous evolution in algorithmic complexity, with each module attempting to resolve the specific structural failures of the previous one. Specifically, statistical density checking (EmbedKDECheck) was introduced to overcome the rigid phrasing constraints of rule-based NLP (medspaCy); trained natural language inference (AlignScore) was selected to provide the contextual awareness lacking in raw vector space; and finally, targeted in-document extraction (NoteExtractor) was developed to successfully bypass the cross-document register mismatch that defeated the prior three models

9.1 Natural Language Processing

Natural language processing (NLP) is the application of computational, semantic structure applied to sentences, converting unstructured text into a representation which a deterministic algorithm can reason over. Most NLP systems, including the ones used for this paper, follow a broad sequential pipeline of increasingly specific analysis stages. [\(42\)](#) .

- **Tokenisation** splits a stream of text into tokens, such as words, punctuation and other meaningful units.
- **Sentence segmentation** then determines sentence boundaries. Different NLP systems segment differently; where a clinical system treats a full stop as part of an abbreviation (e.g. "DR." or "P.M.H"), a general NLP trained on an ordinary prose treats it as a sentence break. This difference demonstrates that a general NLP is not universal across domains, where a clinical text needs a clinical domain sentence decorator [\(42\)](#) .
- **Part-Of-speech (POS) tagging** assigns each token a grammatical category, such as noun, verb, adjective .etc.
- **Dependency parsing** then connects words to each other using sentence structure rules. For example linking a negation to a finding it modifies or an adjective to a verb it is describing.

- **Named entity recognition (NER)** labels spans of text, such as drug names against a reference model or a trained model.
- **Contextual analysis** identifies which section of a document a given span falls into. Distinguishing for instance the Plan from the Objective section of a SOAP note, since identical words can have different meaning depending on the section.
- **Standardization** takes the raw NER extracted and resolved them to a single canonical concept, correctly matching synonyms and disambiguating terms with several possible meanings to what its intended in the given context.

Each stage in this pipeline is a candidate point at which a deterministic, rule based check can be inserted, the toolkits reviewed below differ mainly in how much of the pipeline they implement.

9.1.1 spaCy as a Foundation

spaCy (43) reframes the NLP pipeline around an object oriented model, in contrast to older libraries such as the Natural Language Toolkit (NLTK), which treat text processing as operations on a plain lists of strings (44). This distinction matters, as spaCy's tokenization is non destructive: all whitespace and punctuation are preserved as part of the token stream rather than discarded, so the original text can always be exactly reconstructed from the processed document (42). This is what makes it possible to take a labeled span, that a checker has flagged, a drug name or a negated finding, and locate it back in the original note or transcript with certainty, rather than an approximate rematch against normalized text. SpaCy is also built with Cython, an optimizing compiler that turns Python into efficient C or C++, giving it better throughput than a pure Python library such as NLTK. Because spaCy's API is also easier to extend due to object-oriented programming (OOP), it has become the substrate for several domain specific toolkits, including the two clinical/biomedical variants medspaCy and scispaCy.

9.1.2 scispaCy

scispaCy (45) retrains spaCy's POS tagger, dependency parser, and NER on biomedical text, principally the GENIA corpus of PubMed abstracts, and adds biomedical tokenization rules as well as an approximate nearest neighbor UMLS candidate generator built over Term frequency and inverse document frequency (TF-IDF) character trigrams for 2.78 million UMLS concepts. It ships as two packages, `en_core_sci_sm` and `en_core_sci_md`, differing in vocabulary size and the presence of word vectors.

scispaCy is fast, competitive with pipelines written in C++ and Java specifically for production use (45) and it is a strong general baseline across nine biomedical NER benchmarks spanning chemicals, genes, cell types, and disease names. Because it trains directly on in domain data, its own sentence segmenter also outperforms a general web text model on PubMed abstracts, removing the need for a separate rule based sentence splitter for that specific domain.

However scispaCy's models are trained on published biomedical literature, not clinical narrative, two different sub languages. Its authors are explicit that, at the time of publication, scispaCy has no built in negation, or context handling, naming this directly as future work (45). That absence is exactly the gap medspaCy closes.

9.1.3 NegspaCy

NegspaCy (46) is a spaCy pipeline component implementing the NegEx algorithm (47), one of the earliest and most widely replicated rule based methods for detecting negated findings in clinical text. It works from trigger phrase-lists of pseudo negations (phrases that resemble a negation but are not, e.g. "not ruled out"), preceding and following negation triggers, and termination phrases that end a negation's scope (e.g. "but") and flags any named entity that falls inside a trigger's window as negated.

It is lightweight and quick to add onto any existing spaCy NER pipeline with no retraining required, and it directly fills the negation gap scispaCy leaves open.

Because it detects negation alone on a fixed window around each entity, it cannot represent the other axes the ConText algorithm provides in a single pass, temporality, certainty, and experienter (48) and a window/trigger list approach is more prone than a syntax aware method to mis-scoping a negation across a clause or sentence boundaries. Such like boundaries in multi party transcripts. This project's own condenser comparison Table 2 found negspaCy the most aggressive of the four condensers tested,

removing 35.9–36.8% of words on average against 18.9–25.3% for the others consistent with a window based trigger firing more liberally than a more context sensitive rule.

9.1.4 MedspaCy

MedspaCy (42) is the clinical specific spaCy toolkit. Where ScispaCy targets published biomedical literature, MedspaCy targets clinical narrative itself, whose grammar its authors describe as telegraphic, with omitted subjects and long value lists for present or absent findings and medications. It bundles: a tokenizer with custom punctuation/whitespace handling for clinical shorthand; PyRuSH, a rule based clinical sentence segmenter, since a general purpose segmenters trained on ordinary prose perform poorly on clinical text; a rule based sectionizer that partitions a document into sections such as Subjective, Objective, Assessment, and Plan; a target matcher for concept extraction; an adapted QuickUMLS component for UMLS concept mapping ; and an implementation of the ConText algorithm (48)) for contextual analysis, asserting negation, temporality, certainty, and experienter attributes on every extracted entity.

MedspaCy is built specifically for the structural idiosyncrasies of clinical documentation, and it has been operationally validated at real scale: the U.S. Department of Veterans Affairs has run medspaCy pipelines in production for syndromic surveillance since 2019, and for COVID-19 case identification (82.5% precision / 94.2% recall against manually reviewed charts) and screening documentation, together processing tens of millions of clinical documents (42) . This track record is the reason it was adopted as this project’s default condenser as well as it showing the most positive result in Table 2.

However it is a rule-based system at its core, so its accuracy on any new note style is only as good as the rules configured for it, its sectionizer, in particular, must be told what a note’s section headers look like. A header format it has not been told about (e.g. “Imp-” instead of “Imp:”) will simply not be recognized.

9.1.5 UMLS Concept Mapping: MetaMap and QuickUMLS

Both scispaCy’s candidate generator and medspaCy’s concept extraction component resolve spans of text to the Unified Medical Language System (UMLS) Metathesaurus (49) , the standard reference vocabulary for biomedical concepts. Two tools dominate this specific mapping step.

MetaMap (50) , developed at the U.S. National Library of Medicine, is the classical approach: it parses text into simple noun phrases with the SPECiAlIST minimal commitment parser, generating lexical variants (synonyms, acronyms, derivational forms) for each phrase word, retrieving every Metathesaurus string containing at least one variant, and scores each candidate against the original phrase using a weighted combination of four metrics: centrality, variation, coverage, and cohesiveness, before returning the highest scoring complete mapping. It remains a foundation of NLM’s own indexing infrastructure, but its multi stage linguistic analysis carries a high computational cost.

QuickUMLS (51) was selected by medspaCy’s own authors specifically because its speed and matching accuracy are comparable to systems such as MetaMap at a fraction of the computational cost (42) : it performs efficient approximate dictionary matching using an implementation of the SimString algorithm (52) rather than MetaMap’s full parse and score pipeline. This project follows medspaCy’s choice; the shared matcher used by every type of condenser and SOAP checker in this codebase is a QuickUMLS instance. Beyond medspaCy itself, QuickUMLS’s speed/accuracy trade off has made it a common default for rapid UMLS concept mapping wherever a full MetaMap install is impractical, such as need for quick SOAP checks in a restricted hardware systems, where this project deploys.

9.2 Proposed Use of NLP for SOAP checker

9.2.1 Selecting a condenser: cosine and ROUGE-1 evaluation

Before any SOAP checker instances were built, this project first needed to establish which of the four toolkits, QuickUMLS, medspaCy, negspaCy, and scispaCy, each wrapped in this codebase as a “condenser” that strips a raw transcript down to its asserted clinical content before any downstream comparison, actually preserves the clinical content that mattered, and which evaluation metric could be trusted to answer that question. Eight candidate metrics across three families were tested: KDE-based groundedness/omission scores, cosine similarity, and ROUGE-1. By condensing eleven transcripts by hand to a target length and comparing each metric’s score for the hand condensed version against a random word deletion algorithm cut to the identical length. A trustworthy metric should always prefer the semantically

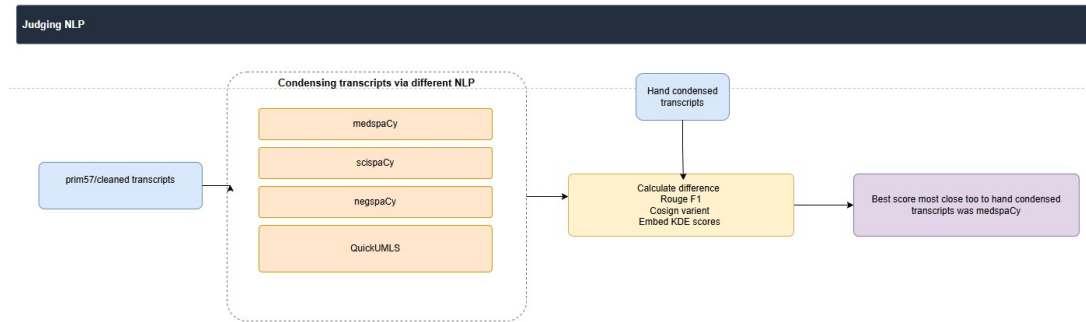


Figure 6: Process of choosing best NLP for this task

meaningful condensation over the random one. Checked file by file rather than as a single averaged number, ROUGE-1 recall, cosine recall, and cosine F1 got this comparison right in all eleven of eleven files, with no exceptions. The KDE-based groundedness metric, showed higher scores in the random deletion transcripts. By this metric’s own convention (8) , a higher score means more omission risk detected, and a very low or negative diff means little to none. So a metric working correctly should show a high rise when real content is deleted, exactly the validating behaviour seen in the files where KDE omission got the direction right. KDE’s underlying approach is fit for this specific question : it asks whether the condensed transcript content falls inside the density of the full transcript’s own embedding space, a full space versus one of its own sub spaces rather than the harder comparison of scoring how well a compressed summarized SOAP matches a transcript, which is discussed later in this thesis.

On this evidence, ROUGE-1 recall and cosine F1 were adopted as the two metrics to trust for evaluating condenser quality going forward chosen together specifically because they are methodologically independent (one an exact lexical overlap measure, the other embedding based), so agreement between them is a triangulation rather than two views of the same computation, while the KDE based scores were kept to identify groundedness comparisons. Applied to the four condensers over the same 11 files compared against the hand curated random deletion control transcripts, all four preserved genuine semantic content by this standard every condenser scored a positive ROUGE-1 F1 difference against its random deletion control, with cosine F1 difference sitting close to its zero ceiling in every case while differing substantially in how aggressively each condensed the text:

Condenser	Words removed	ROUGE-1 F1 diff.	Cosine F1 diff.	KDE groundedness diff. ¹
QuickUMLS	18.9–20.7%	+0.01	≈0.00	0.42
medspaCy	23.2–25.3%	+0.02	≈0.00	0.06
negspaCy	35.9–36.8%	+0.03	≈0.00	0.44
scispaCy	7.0–7.4%	≈0.00–0.01	≈0.00	0.79

Table 2: Performance and compression characteristics of four clinical NLP condensers evaluated against a random deletion baseline across eleven transcripts. Words removed shows the percentage of text stripped; ROUGE-1 F1 diff. measures superiority over random word removal in preserving key terms; Cosine F1 diff. evaluates semantic similarity; and KDE groundedness serves as a secondary density based cross check¹.

This comparison is best read as validating the metrics rather than crowning a single winning condenser outright. By ROUGE-1/cosine F1 alone, negspaCy’s more aggressive filtering is not obviously the wrong choice. What tipped the decision toward medspaCy as this project’s default condenser, was that the more aggressive filtering of NegspaCy came at the cost of worse groundedness. MedspaCy had the best groundedness and comparable ROUGE-1 score. Combined with its clinical domain design and its built in section and contextual assertion handling neither of which negspaCy or QuickUMLS provide on their own, medspaCy was therefore adopted as the pipeline’s default condenser.

9.2.2 The medspaCy based checker

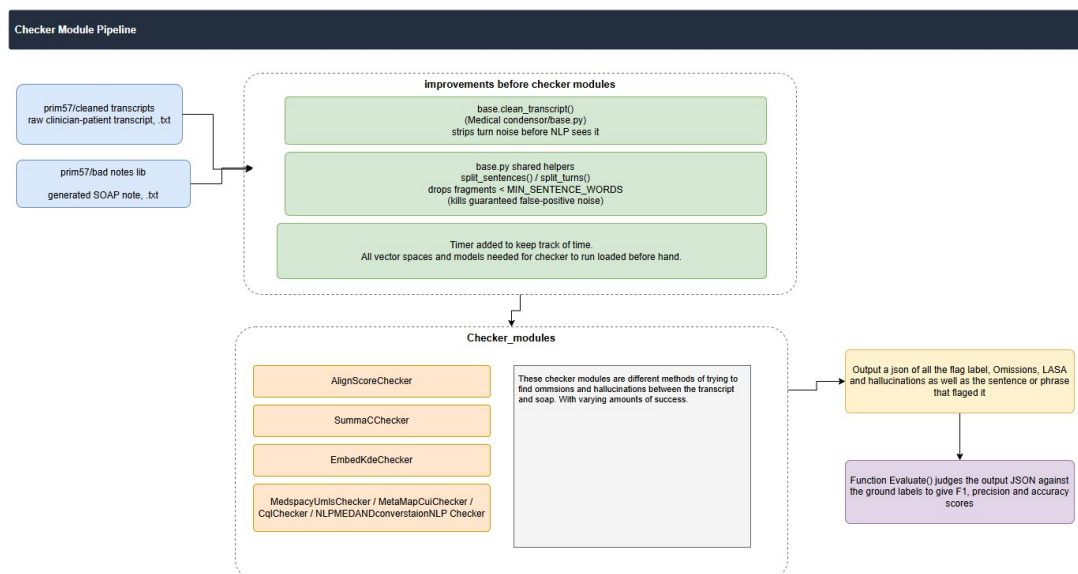


Figure 7: How the checker modules are evaluated

The first SOAP checker module built on top of medspaCy follows a three step design: extract every concept the shared QuickUMLS matcher finds in both the transcript and the SOAP note, scoped to UMLS semantic types covering diseases/findings (T047, T033, T184, T191), procedures (T059–T061), and drugs (T121 Pharmacologic Substance, T195 Antibiotic, T200 Clinical Drug) (51); run medspaCy’s ConText component over every extracted span to tag its assertion state (negated, uncertain, historical, hypothetical, or family-associated); then compare the transcript and SOAP concept sets by their UMLS Concept Unique Identifier (CUI) to raise one of three deterministic judgments. A concept in the transcript with no matching CUI anywhere in the note is an omission, a concept in the note with no matching CUI anywhere in the transcript is a hallucination, and a concept present in both documents whose assertion state never agrees between any transcript mention and any note mention is a status flip. Scored over the full prim 57-file evaluation dataset, this checker performs poorly: 5.5% precision and 16.7% recall overall ($F1 = 8.3\%$) Table 5, with hallucination detection reaching 50% recall at only 5% precision, and omission detection catching nothing at all (0% precision and recall).

9.2.3 Error Analysis & Failure Modes

This proves NLP alone is not enough for this transcript, SOAP checker. The gap between this result and medspaCy’s own validated deployments is instructive rather than surprising. Those deployments, chief complaint triage strings, COVID-19 screening notes (42), are themselves written as clinical text: templated, single authored, and already conforming to the section and telegraphic grammar conventions medspaCy’s rules were built against. A transcript is neither. It is two party, spontaneous, spoken dialogue, transcribed with disfluencies, false starts, and a doctor questions separated from a patient one word answers by a turn boundary. A plain concept extraction step has no way to bridge on its own the context started from one party and ended by another. Comparing a note’s concept set against a transcript’s concept set by plain CUI set difference assumes the two documents are drawn from the same register of language; they are not, and that assumption breaks in at least three concrete ways confirmed directly against this dataset over the course of this project. Ordinary conversational words coincidentally collide with real UMLS concepts under exact or near exact matching (“well,” “start,” and “right” are themselves valid, if clinically generic, Metathesaurus entries); a single clinical fact is frequently split across a doctor’s question and a patient’s short reply, rather than stated once, in one utterance a section aware extractor can read as a whole; and near synonymous phrasing on the two sides of the same fact (a patient saying “loose stool” against a note recording “watery stools”) can resolve to two different CUIs under approximate matching, so a real, correctly documented fact still looks like a mismatch by concept identity alone. None of these are failures of medspaCy specifically, they are the predictable

consequence of pointing a pipeline tuned for one register of clinical language at a fundamentally different one: a spoken, and non deterministic one. A written note’s phrasing is fixed once it is written; a spoken transcript’s is not, and a checker built only from general purpose NLP components has no way to know in advance which of many equally valid spoken phrasings a given fact might take. It cannot guess at that mapping and still call itself deterministic, it has to be told, explicitly and in advance, every shape a fact is allowed to take before it can be trusted to compare it against the note at all. This is the motivation for the additional, transcript-specific handling layered into this project’s other SOAP checkers beyond what medspaCy provides.

9.3 KDE-Based Semantic Density Checking (EmbedKDECheck)

Given the caveats of relying on pure NLP, a spoken clinical consultation admits hundreds of colloquial, connotative ways of stating the same clinical fact, a rigid, rule based parser tuned to one phrasing cannot be relied on to recognize all of them. Where possible deterministic methods sets out symbolic, rulebased NLP, this section sets out a second, purely statistical, embedding space method that makes no assumption about surface phrasing at all. Kernel Density Estimation (KDE) omission checking, EmbedKDECheck (8) , is a deterministic algorithm that projects token embeddings into a reduced dimensional continuous semantic space via Principal Component Analysis (PCA), then measures how well a candidate document’s tokens are represented by the density of another document’s tokens in that same space.

9.3.1 How EmbedKDECheck works

EmbedKDECheck is a computationally frugal, black box, reference free method for evaluating factual consistency by detecting omissions in text summaries or reformulations. It uses Kernel Density Estimation a non parametric method to estimate the probability density function of token embeddings in a continuous semantic space, without ever training a model of its own or requiring a human written gold-standard reference (8) .

$$p(x) = \frac{1}{nh} \sum_{i=1}^n K\left(\frac{x - x_i}{h}\right) \quad (1)$$

where:

- n — the number of tokens in the reference (output) text.
- K — a Gaussian kernel.
- h — the bandwidth parameter, controlling the smoothness of the estimated density (a larger h gives a smoother, more generalized distribution; a smaller h is more sensitive to local variation but can introduce noise).

Using these estimated densities, the local omission score (OM) for a given token is the ratio of the reference document’s own lowest self density to that token’s density under the fitted KDE:

$$OM(w) = \frac{\min_{v \in \text{Reference}} p(v)}{p(w)} \quad (2)$$

The global omission score for an entire document is the maximum token level score across every token in the candidate text:

$$OM_{\text{doc}} = \max_{w \in \text{Candidate}} OM(w) \quad (3)$$

This formulation ensures that if even a single critical concept from the candidate text falls into an extremely low density region of the reference’s semantic space, it yields a high omission score, flagging a significant omission. The method is deliberately worst case sensitive rather than averaging the mismatch away.

What it needs to run

- An input text and an output text : No gold standard reference summaries and no access to an LLM’s internal token probabilities are required, which is what makes it usable against any ambient scribe as a true black box, vendor model included.

- A word level embedding provider. The method is language agnostic and needs only some way of embedding tokens into a shared vector space; the original paper uses FTW2V, a FastText+Word2Vec hybrid fine tuned on a 32 million word clinical French corpus from the collaborating hospital (8) .
- Dimensionality reduction (PCA). KDE degrades in high dimensional spaces, so PCA compression of the embeddings is required before density estimation, both for stability and for speed.
- Tuned hyperparameters : A detection threshold, the number of retained PCA components, and the KDE bandwidth, each tuned on a validation set to balance precision and recall.

9.3.2 Why This Fits an Ambient AI Post-Market Check

The method’s own published benchmark, shown in Table 3 is directly relevant to the deployment constraints of restricted hardware in a hospital setting where a monitoring layer that has to run continuously, in real time, on the clinician side of the pipeline, without adding cloud dependency to an already privacy sensitive workflow.

Model	Recall	Precision	F1	Est. compute cost (TFLOPS)
EmbedKDECheck	0.88	0.93	0.91	0.11
GPTScore	0.80	0.79	0.80	2.14
G-Eval (GPT-3.5-Turbo)	0.98	0.46	0.63	513.81
ChainPoll (GPT-3.5-Turbo)	0.97	0.67	0.51	770.73
SelfCheckGPT (GPT-3.5-Turbo)	0.79	0.77	0.78	1,545.40
G-Eval (GPT-4)	0.99	0.68	0.80	2,936.04
ChainPoll (GPT-4)	0.99	0.68	0.80	4,404.16
SelfCheckGPT (GPT-4)	0.86	0.84	0.85	8,804.03

Table 3: Reported scores of KDE against common models on the binary classification of if there is an omission or not in a text (8)

A 0.11-TeraFLOP, 2.4 GB-RAM footprint (8) is small enough to run on standard hospital CPU hardware. Because no patient transcript or note ever has to leave the local network to reach a third party model API, the check does not introduce the GDPR/data residency exposure this which is a NHS concern, the same reason MHRA’s AI airlock sandbox exists to interrogate. A sub second, CPU only check per sentence is cheap enough to run on every consultation, continuously, rather than being reserved for periodic batch audits making it easier to see issues per consultation without extra work.

9.3.3 Proposed Use: Bidirectional, Per-Sentence Density Checking

The published method by Oukelmoun et al. (8) ,scores in one direction only, at whole document level. It fits a KDE on an LLM generated summary’s word embeddings, then scores every word of the original source document against that density map, catching omissions the source content is missing from the summary (8) . It identifies a binary output if there is an omission Table 3 but not what the specific omission or how many there are. This paper’s implementation generalizes this in two ways to fit a SOAP note checking pipeline rather than a single full document summarization task:

- **Per sentence, not per document.** Each candidate sentence’s score is the aggregate of its own word’s token level scores, the per sentence analogue of Equation (3), flagged individually if that aggregate crosses a threshold, rather than one score for an entire document. This is to find exactly which sentence is flagging.
- **Bidirectional.** A KDE fitted on the transcript’s word embeddings scores each SOAP note sentence for hallucination; a KDE fitted on the SOAP note’s word embeddings scores each transcript sentence for omission. This is an attempt to “extend to other forms of hallucination” direction the original paper names as future work (8) .

Concretely, checking one SOAP sentence against the whole transcript works as follows: every word in the transcript is embedded and PCA reduced, and a KDE is fitted over that entire reduced point cloud, the transcript’s own semantic space, but restricted to whatever register and vocabulary a spoken consultation actually used. The SOAP sentence’s own words are embedded, projected into the same PCA space, and scored under that fitted KDE.

- **A high hallucination score** means the SOAP sentence’s words sit in a region of space unlike anything the transcript’s density map covers. The sentence describes clinical content with no semantic neighbour anywhere in what was actually said, the embedding space signature of a fabricated finding, drug, or diagnosis.
- **A high omission score** is the mirror case in the other direction: a transcript sentence whose words sit far outside the density the SOAP note’s own vocabulary. Content that was said, but that the note’s semantic space gives no account of at all.
- **A low score**, in either direction, means the opposite: the sentence’s words land inside a dense, populated region of the other document’s embedding space. Semantically unremarkable relative to what the reference document already contains, and therefore not flagged. The method has no notion of a score being “too low” to worry about, only high scores are informative, since density is being used purely to detect the absence of support, not to grade how well supported a sentence is.

9.3.4 Results

Evaluated against this project’s own hand-labeled hallucination/omission ground truth (a 3-file slice of the 57-file prim1, prim10, prim11 and the full set), the checker’s real performance falls well short of the published benchmark [Table 4](#)

KDE run	<i>n</i> files	Precision	Recall	F1	Avg. s/file
3-file sample, overall*	3	12.1%	50.0%	19.5%	6.34s
3-file sample, hallucination only	3	24.0%	80.0%	36.0%	—
Full 57-file, overall†	57	4.4%	44.7%	8.0%	4.23s
Full 57-file, hallucination only	57	18.7%	55.1%	27.9%	—
Full 57-file, omission only	57	0.0%	0.0%	0.0%	—

Table 4: Observed scores from the KDE proposed solution, showing hallucinations work to a degree and omission do not as much

The omission direction is the more informative failure: none of the flags were correct, on a candidate pool built almost entirely from ordinary spoken dialogue turns. A representative sample from the evaluation log:

```
Flagged as omission (prim1.txt) — none are clinical content:
[FALSE] omission: d: Sorry to hear that.
[FALSE] omission: d: How's that sound?
[FALSE] omission: d: Do you have any questions for me?
[FALSE] omission: d: And do you drink much in the way of alcohol?
```

Figure 8: Example flagged omissions

These are conversational filler, exactly the content a clinician would never expect, or want, reproduced in a SOAP note. The checker has no way to distinguish them from a genuinely omitted clinical fact; density based scoring alone cannot tell “this doesn’t resemble the note because it’s clinically irrelevant” apart from “this doesn’t resemble the note because it was dropped.”

9.3.5 Error Analysis & Failure Modes

The token level score has an extreme, unstable dynamic range [Equation \(3\)](#). Raw ratio spans roughly 10^{-5} to 10^{34} on this project’s own word embeddings, a single word’s KDE density can land numerically

indistinguishable from zero. Under Oukelmoun et al. paper’s (8) own max aggregation Equation (2), one outlier word is enough to decide an entire sentence’s score regardless of every other word in it, and was the direct cause of an early over flagging failure mode during development; this implementation aggregates by mean instead, and computes the whole ratio in log space with a hard clip before exponentiating, specifically so one underflowing word can no longer silently poison the sentence score to inf .

Three iterations of the method was developed to make KDE usable in this context. Equation (3) numerator , “the reference’s own minimum density”, turned out to be the single sensitive part of the whole method on a document as short as one transcript or SOAP sentence. Three attempts on the same file (prim1.txt : 128 claims):

Attempt 1 : literal in sample minimum, threshold = 1.0 : 112 / 128 claims flagged (87.5%) Scoring each reference word under a KDE that includes its own kernel contribution inflates its density, since the point boosts its own score. A genuinely well matched candidate word never gets that same self-boost, so almost anything scores below the inflated floor, combined with max aggregation, over flagged nearly everything.

Attempt 2 : leave one out strict minimum 0 / 128 flagged (0%) Removing the self inflation fixed the direction of the bias but overcorrected it entirely: on a reference set of only a few dozen words (a transcript or SOAP sentence, not the paper’s own hundreds of words medical reports), the strict minimum is dominated by whichever single reference word happens to be the most unusual one, in or out of sample and that one word sets the anchor so low that nothing exceeds it.

Attempt 3: leave one out, 10th percentile 64 / 128 flagged (50%) Anchoring on the 10th percentile of the leave one out scores rather than the strict minimum keeps the “near the floor” intent without being hostage to one outlier reference word. This is the version behind the results in Table 4 a workable middle ground, not a calibrated one: THRESHOLD and SELF_DENSITY_PERCENTILE remain starting points rather than values swept against labels at any scale beyond this single file.

The reference corpus is the wrong shape for the method’s own assumption

EmbedKDECheck’s density estimate is only as meaningful as the reference document’s own internal consistency. The original paper’s reference documents are whole medical reports, written once, in one register, by one author such like what medspaCy is made for. A transcript is neither: it is dozens of short, spoken, two party conversational turns, most of which, by design, not by omission, are never meant to appear in a note at all (greetings, rapport building, repeated clarifying questions). Scoring every one of those turns against a short, condensed SOAP note’s density map with the transcript as the output, all but guarantees most of them will land in a low density region regardless of whether any clinical content was actually lost, which is the direct, mechanical cause of the omission direction’s 0% precision in Table 4. The embeddings are a bigger domain mismatch than the paper’s own. The paper’s FTW2V is fine tuned on a 32 million word clinical corpus drawn from the same hospital and, by extension, the same documentation register as its evaluation data (8) . That fine tuned model is not available here; this implementation concatenates two generic, English, non clinical pretrained embeddings (FastText and Word2Vec) instead. Every word that either model fails to recognize is dropped from the candidate pool entirely rather than approximated, and neither model has ever seen the specific register, spoken UK primary care consultation, either the transcript or the note is actually written in.

9.3.6 Possible Fixes for Further Research for using KDE in this context

- **Fine tune the embeddings on this project’s own corpus**, the same way the original paper fine tuned FTW2V on its hospital’s clinical French, closing the largest single domain gap identified above, rather than relying on generic pretrained English vectors
- **Pre filter the transcript before scoring the omission direction**, reusing the medspaCy condenser already validated in Table 2 to strip conversational, non clinical turns before they ever reach the KDE, directly targeting the failure mode demonstrated in Table 4 example.
- **Investigate a normalized or rank based aggregate in place of the raw mean**, since Equation (2) ratio remains extreme even after log space clipping, a small number of very high scoring words can still dominate a mean the way they dominated the original max.

9.4 AlignScore: Trained Alignment Model

KDE based density checking as a route to a deterministic factual consistency check between SOAP and transcript avoiding the rigidity of a purely rule based NLP parsers has a structural weakness in that where

raw embedding density estimate has no notion of task. Ordinary words correlate with each other in the embedding space regardless of whether they are making the same clinical claim, a transcript sentence can sit close to a SOAP note’s density map, or far from it, for reasons that have nothing to do with whether the same fact is actually being stated. AlignScore takes a different route to the same deterministic goal: rather than measuring raw semantic proximity, it uses a model explicitly trained to recognize whether one piece of text’s information is supported by another, by unifying seven established language understanding tasks into one alignment function (53) . It remains deterministic in exactly the sense: once trained, its weights are fixed, so a given claim/context pair always produces the same score nothing about the check itself is sampled or regenerated at inference time.

9.4.1 How AlignScore works

AlignScore is a holistic, generalisable metric for evaluating the factual consistency of a generated text (the claim) against an input source (the context) (53) . Traditional evaluation metrics are often limited because they rely on narrow functions trained for one specific task ,Natural Language Inference (NLI) or Question Answering (QA) , which hinders their ability to generalise across domains or document lengths. AlignScore instead frames factual consistency as a single, unified information alignment problem, trained on a large and diverse corpus so that the same function can evaluate contradictions and hallucinations alike, across tasks it was never directly trained on.

1. The alignment function: A claim is considered “aligned” with a context if every piece of information in the claim is present in the context and does not contradict it. AlignScore maps any such text pair to a single alignment score. (53)

2. Unified training: The underlying model is trained on 4.7 million examples aggregated and reformatted from 15 datasets across 7 distinct tasks: NLI, QA, paraphrasing, fact verification, information retrieval, semantic similarity, and summarization. Tasks that do not naturally fit a text pair format, QA and information retrieval, are converted into declarative sentences by a sequence to sequence model before training, so every task is ultimately reduced to the same claim/context comparison. (53)

3. Three way classification head: The model carries three output heads (binary classification, 3-way classification, and regression) to preserve every source dataset’s original label format during training, factual consistency scoring specifically uses the 3-way head’s probability of the ALIGNED label. Which is the configuration the paper’s own ablations found consistently strongest. (53)

4. Context and claim splitting: Long source documents routinely exceed a standard transformer’s input limit (RoBERTa’s 512 tokens), which silently truncates anything beyond it. AlignScore avoids this by splitting the context at sentence boundaries into coarse, roughly 350-token chunks, while the claim is split into individual, fine grained sentences by a sentenciser. (53)

5. Evaluation and aggregation: Each claim sentence is scored against every context chunk, and only the highest score across all chunks is kept for that sentence identifying whichever chunk most strongly supports it. These per sentence maxima are then averaged across the whole claim to produce the final score, so one badly matched context chunk cannot drag down a sentence that is genuinely well supported elsewhere in the document, and one inconsistent sentence cannot single handedly dominate the whole claim’s score either (53) .

Align score excels at:

- **Breadth of error types;** contradictions, hallucinations, and unfaithful term substitutions, across generation tasks, not just one.
- **High performance at a compact size:** AlignScore large (355M parameters) matches or outperforms evaluation methods built on ChatGPT and GPT-4, at a small fraction of their size. (53)
- **Zero-shot generalization:** state of the art results on 19 evaluation datasets never seen during training of the alignment function itself.
- **Long-document context:** 350 token coarse chunking retains paragraph level semantic relationships that a purely sentence level metric would lose.

9.4.2 Why This Fits an Ambient AI Post-Market Check

AlignScore sits between the two extremes : a prompted GPT-4-class judge (heavy, cloud-dependent, and non deterministic, the exact property the deterministic checker exists to avoid) and the pure statistical density estimate of KDE. At 125M parameters, AlignScore-base is small enough to run entirely on CPU,

this project’s own implementation sets device=“cpu” outright, with a downloaded checkpoint held locally, so no transcript or note ever needs to leave the hospital network to be scored, the same privacy argument makes for KDE checking. This is not as cheap as KDE’s pure vector density arithmetic: a transformer forward pass over a few hundred tokens is still orders of magnitude more compute than scoring a handful of pre-computed embeddings under a fitted kernel, even at 125M parameters. But relative to the GPT-4-class alternatives whose compute cost was tabulated in Table 3 thousands of TeraFLOPS per call a 125M parameter local model run on standard hospital CPU hardware is a comparable order of magnitude saving. Without AlignScore’s own generalisation results being sacrificed. It is, a deliberate mid point: heavier than KDE, but trained specifically for this task rather than borrowing a general purpose density estimate; lighter than an LLM judge, and, critically, deterministic and local in a way an API hosted model can never fully guarantee.

9.4.3 Proposed Use: AlignScore as a Per-Sentence Hallucination Check

This paper’s implementation uses AlignScore base (roberta-base) with Zha et al. paper’s (53) own recommended nli_sp evaluation mode, the 3-way classification head, run entirely on CPU. Each SOAP note is split via a custom splitter built for the SOAP. Where newlines, fullstops and minimum sentence length of 2 words determines sentence spans. Oppose to the original sentencizer which only takes valid full stops. This edit to the algorithm shows a x5 improvement in application Table 5. Included is AlignScore’s own internal chunking handling any context longer than one 350 token chunk. A sentence is flagged as a hallucination if its alignment score falls below a threshold of 0.25 and later adjusted to 0.3 ,when a new sentence segmenter algorithm implemented, calibrated on six hand picked candidate phrases from a single file (three test paraphrases scoring 0.302–0.863, three unrelated/false phrases scoring 0.077–0.211), with 0.25 chosen initially to sit cleanly in the gap between the lowest true score and the highest false one. The later adjustment to 0.3 can see a x1.25 increase seen in Figure 10. Currently implemented, this checker runs in the hallucination direction only SOAP sentence as claim, transcript as context the symmetric omission direction (transcript sentence as claim, SOAP note as context) is not yet scored as additional development is needed to sentencize the transcript properly to account for the cross sentence boundaries for negation, temporality, certainty, and experienter described in (48)

Three Edits to the original algorithm was implemented in attempt to specify the task to clinical transcript and Soaps.

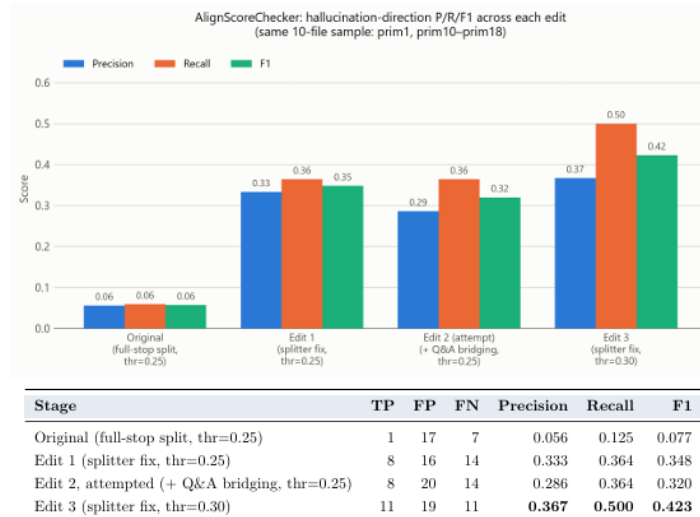


Figure 9: Result on scores for the subset of 10 from prim57 dataset after each edit.

Edit 1: a clinical note aware sentence splitter

Extending the original splitter via full stops to include newlines as well as on a forward slash specifically when it precedes a recognized clinical field header, PMH:, DH:, SH:, Plan:, and similar, so “PMH: Asthma / DH: Inhalers / SH: works as an accountant.” now splits into three separate claims instead of surviving as one fact blob. The slash rule is deliberately narrow: this dataset also uses “/” to join two findings under one shared negation, and a blind split there would strand “chest pain” without the “No”

negating it, silently flipping a true negative claim into a false positive of the opposite kind. Restricting the rule to slashes immediately followed by a field label avoids that failure mode while still catching the original issue of a fact blob due to the original sentencizer. The effect on real numbers is the largest single change of the three edits: F1 moves from 0.077 to 0.348 [Figure 9](#) on the hallucination direction, with both precision and recall improving, not a precision/recall trade, a strict gain in both.

Edit 2 (attempted): bridging implicit doctor-question/patient-reply pairs

The dominant remaining false positive class after Edit 1 was a different problem: true, correctly summarised SOAP facts that are negative or terse answers to a doctor’s question in the transcript, “any arm weakness?” / “No.”, never appear anywhere in the transcript as an actual declarative sentence after run through the sentencizer for AlignScore to match the SOAP claim “No arm or leg weakness” against, similar to the reason why omissions was omitted in development. The attempted fix, a Adjacency pair Context Propagation algorithm, was used to resolve each doctor question/patient reply pair into an explicit synthetic assertion (“No arm weakness.”) and append it to the transcript before it is used as AlignScore’s context. Implemented and tested on the same 10 files; did not improve the result: F1 fell slightly, from 0.348 to 0.320, driven entirely by four new false positives with no corresponding new true positives (TP and FN both held exactly constant at 8 and 14). A false positive negation claim on prim13.txt was correctly resolved, but new false positives appeared on three other files that had been clean before. The most likely cause: appending every bridged assertion to the end of the whole transcript lengthens and reshuffles the context used to score every claim in the file, not just the ones the bridging was meant to help, so the fix’s noise (perturbing unrelated claims’ due to inconsistent question and answer matching) came out roughly proportional to its signal. This reads as a real but poorly localised implementation of an idea, not evidence the idea itself is wrong but further implementation of a adjacency pair Context Propagation algorithm is required.

Edit 3: re-calibrating THRESHOLD after the splitter fix The initial THRESHOLD = 0.25 as calibrated on the wrong task (six hand picked clean phrases, not real bundled SOAP sentences), Edit 1 changed what a “claim” looks like, so the old threshold was never re validated against its own fix. A sweep was run: every claim across the 10 file sample scored once by AlignScore (a single model pass, not one pass per threshold candidate), then 14 threshold values tried against those cached scores.

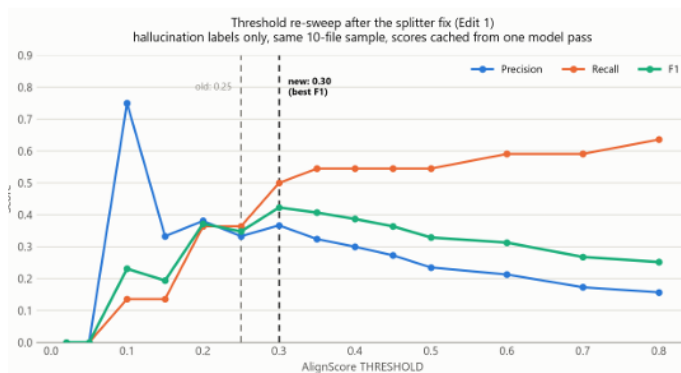


Figure 10: Finding best threshold

The peak sits at THRESHOLD = 0.30, F1 rises from 0.348 to 0.423, and , no trade off recorded , since both precision (0.333→0.367) and recall (0.364→0.500) rise together. [Figure 9](#) .

9.4.4 Results

Evaluated against this project’s hand labeled ground truth e 57-file dataset , the checker performs poorly markedly worse than the published benchmark cited in (8) but after edits better than the KDE checker’s own weak hallucination direction result in [Table 5](#):

This dataset’s transcripts are diarised turns marked only by a leading d:/p: tag and a newline alignscores tokenization and sentencizer has no notion of, since it was trained on continuous written prose and treats a newline as ordinary whitespace, not a boundary. Confirmed directly across all 57 cleaned transcripts: 126 of the “sentences” the sentencizer produces span three or more original transcript lines, frequently crossing speaker turns in the process.

p: OK.

d: Any of those symptoms, or you start being violently sick with it, then you know, you

would need to take you to A and E.

p: OK.

d: But if not, wait for the appointment for the doctor to examine you um and we'll take it from there.

The two instructions this glue together are conditioned on opposite scenarios, go to A&E *if* red flag symptoms appear, wait for a routine appointment *if* they do not, and once merged they sit inside the same ~350-token context chunk as an undifferentiated block. A SOAP claim like “Advised to attend A&E if red flag symptoms develop” is now scored against a chunk where that instruction is not cleanly isolated but run together with its own conditional opposite, exactly the kind of coherence loss KDE and medSpacy already diagnosed on the claim side, here inflicted on the context side. A “splitter fix” claim is accordingly only half true: the SOAP-side instance splitter via Edit 1 is fixed, but the transcript side instance, arguably the more consequential one since every claim in a file shares the same handful of context chunks, has not been addressed.

9.4.5 Error Analysis & Failure Modes

$F1 = 0.423$ is a roughly $5.5\times$ improvement over the original 0.077, but it is still a poor result, the checker still misses more than half of real hallucinations (11 of 22 FN) and is wrong more often than right when it does flag something (19 FP against 11 TP) [Figure 9](#). This is also ignoring omission labels.

- **Implicit Q&A evidence is still mostly invisible to the model.** Edit 2 confirmed this is real and worth fixing, but also that a whole document fix does not work the model still has to infer, that a doctor’s question plus a patient’s one word reply “No” supports a written negative finding. Since conversation is also not always straight forward a specifically tuned NLP may be needed to pass the transcript before AlignScore is used. Similar issues to KDE and the medSpacy checkers.
- **Numeric substitutions are a structural blind spot** A dosage or frequency edit (e.g. “x60/day” standing in for the true “x6/day”) changes almost nothing in embedding space, the sentence is semantically near identical either way to any model scoring meaning rather than exact values. This method checker will not catch these errors ([53](#)).
- **The register gap.** AlignScore’s 4.7 million example training corpus is broad across tasks (NLI, QA, paraphrase, fact verification, IR, semantic similarity, summarization) but not specifically clinical, and RoBERTa’s own pretraining corpus (web text, books, news) contains essentially none of this dataset’s shorthand (“NKDA”, “PMH:”, “DH:”) or the specific register of a transcribed spoken UK primary care consultation. Where the Zha et al. paper’s ([53](#)) own zero shot results hold up across 19 unseen datasets within its training distribution’s general domain, this project’s data sits further outside that distribution than those benchmark sets did as it encompasses spoken text.
- **Only one direction is implemented** Running the hallucination direction alone means every omission type ground truth label in the evaluation sample is an automatic miss, not because the method failed to detect it, but because it was never asked to look. Making claims from the clinical transcript shares the same issue Edit 2 attempted to fix. Where making proper claims from non natural language is a task to be solved.
- **AlignScore-base, not AlignScore-large** This implementation uses the 125M-parameter base checkpoint for its CPU/hospital-hardware footprint, not the 355M-parameter large checkpoint that Zha et al. ([53](#)) strongest published numbers are measured on. That trade was made deliberately for deployment reasons, but its accuracy cost specifically on this dataset has not actually been measured against the large checkpoint, it remains an assumption, not a verified trade-off.

9.4.6 Possible Fixes for Further Research for using AlignScore in this context.

An untested improvement to the algorithm is fine tuning AlignScore’s alignment head on in domain clinical data, rather than relying on its fixed, general domain checkpoint. Training using real transcripts, SOAP sentence claims, and ground truth **ALIGNED/CONTRADICT** labels derived from corruptions such as hallucinations, negations, entity swaps, and number edits is a good suggestion forward. This approach calls for **continued fine-tuning from AlignScore’s existing checkpoint**, not retraining from scratch, targeting the specific register gap identified (UK primary-care shorthand, spoken dialogue disfluency, and telegraphic note structure), using a small number of in domain examples at a low learning rate to avoid

catastrophic forgetting. If the prim57 dataset proves too limited for safe fine-tuning, a fallback option exists in larger public clinical corpora such as **MIMIC-III/MIMIC-IV** discharge summaries paired with synthetic or extractive summaries.

9.5 SUMMAC

SummaC (Summary Consistency) is a prominent framework for evaluating whether a generated summary is factually consistent with its source document (54). NLI models are, in principle, exactly the right tool for this: they classify whether a premise entails, contradicts, or is neutral to a hypothesis, which maps directly onto asking whether a source document supports a generated claim. In practice, however, off the shelf NLI models perform badly when applied directly to document level summarization, because of a granularity mismatch. NLI datasets are built and trained at the sentence level, while summarization consistency has to be judged at the document level. Feeding a fifty sentence transcript into an NLI model as a single premise causes silent context length truncation or outright failure. SummaC exists specifically to close that gap.

9.5.1 How SummaC Works

SummaC breaks the document and the summary down into smaller units and treats consistency as a block-by-block alignment problem, rather than one enormous premise/hypothesis pair:

Step 1: The source document is split into M sentence blocks (premises) and the summary into N sentence blocks (hypotheses). Every combination (D_i, S_j) is run through an out of the box transformer NLI model, SummaC’s own strongest configuration uses a model trained on MNLI + Vitamin C (“vitc”) producing an $M \times N$ matrix of entailment probabilities.

Step 2 : The score aggregation SummaC offers two ways to collapse that matrix into a single document level score:

- SUMMAC-ZS (zero-shot). For each summary sentence (each column), take the maximum entailment value, the single source sentence that most strongly supports it, then average those per sentence maxima across the whole summary. No training required.
- SUMMAC-Conv (convolutional). The zero shot maximum is highly sensitive to a single outlier score. SUMMAC-Conv instead keeps the full score distribution for each summary sentence, bins it into a fixed size histogram (typically $H = 50$ bins), and passes that histogram through a trained 1-D convolutional layer that compiles it into one robust scalar per sentence, averaged across the summary for the final score. This is the configuration behind SummaC’s published results, outperforming plain document level NLI by a substantial margin (54)

9.5.2 Why This Is a Harder Fit for an Ambient AI Check

Where KDE and AlignScore base fits a CPU bound, real time hospital deployment, SummaC’s own architecture works against that constraint. The $M \times N$ pair matrix means the number of NLI forward passes grows with the product of document length and summary length, not the sum. A full transcript against a multi sentence SOAP note is dozens of sentence pairs, each one a separate transformer inference call. This is the opposite trade off to KDE (cheap arithmetic over pre-computed vectors) and to AlignScore (one model call per claim against pre-chunked context): SummaC pays the model’s inference cost once per cell of the matrix, not once per claim.

Results:

The runtime, not the precision, is the disqualifying number here. A 9% precision Table 5 is poor and not obviously worse in kind. Thirty minutes per file is a difference in kind, not degree, from every other method discussed, it is fundamentally incompatible with “passive, real-time” requirement and with the continuous, per consultation monitoring of this thesis calls for, regardless of what happens to its precision after further tuning. Even with a GPU RTX 3050 Ti the system runs an average 3 minutes per file. Expecting every hospital to have a GPU is a dangerous assumption, and 3 minutes is still a long time. This method also used the zero-shot variant for quicker computation. SUMMAC-ZS’s defining weakness, by Laban et al. (54) own account, is exactly the sensitivity to outlier scores that SUMMAC-Conv’s trained convolutional aggregation was built to fix, delivering the Laban et al. paper’s (54) actual result. The choice of using zero shot means this checker never benefited from the one architectural change the source paper shows fixes precisely the kind of noisy, high variance scoring behind its poor precision here.

However no further test has been done due to the determining factor the algorithm takes too long.

9.6 NoteExtractor: A Lightweight Structured-Summary Layer

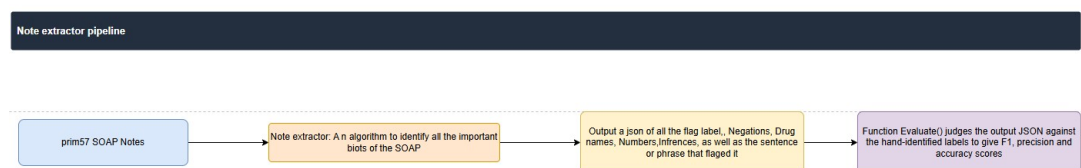


Figure 11: Workflow of the note extractor module.

KDE, AlignScore, SummaC, and the medspaCY checker all address the same question: is this specific piece of generated text factually consistent with its source? NoteExtractor answers a different, narrower question, deliberately: what does this note actually contain? It does not judge whether a negation, a medicine, or an inference statement is correct, only whether it is present, tagged by type. This section sets out why that narrower job is still a useful layer in its own right, reports its performance, and proposes how it could feed into this paper’s existing risk severity model..

9.6.1 How NoteExtractor Works

Plain functions and constants, no checker types classes, no transcript required, NoteExtractor reads a SOAP note on its own and returns every clause that reads as clinical negation, every prescribed medicine it can identify, and every clause that reads as clinical inference or impression, each tagged with the SOAP section it falls in. Negation detection is a cue word regex tuned for this dataset’s terse bullet style; medicine detection reuses the project’s shared QuickUMLS matcher and DRUG.SEMTYPES filter, the same building blocks NLP checker code already uses; inference detection combines Assessment/Impression section membership with hedge-word cues (“likely”, “possible”, “?”).

Method	Answers	Output shape	Transcript?
medspaCy checker (§9.2)	Does the CUI match in the source?	Scalar score	Yes
KDE (§9.3)	Is this claim grounded in the source?	Scalar score	Yes
AlignScore (§9.4)	Is this claim entailed by the source?	Scalar score	Yes
SummaC (§9.5)	Is this claim entailed by the source?	Scalar score	Yes
NoteExtractor (§9.6)	What does this note contain?	Tagged clauses	No

9.6.2 Results

Scored against a hand built label set covering all 57 real notes (independent of the labels used to score other checkers), NoteExtractor performs substantially better than any of the verification checkers with 552 true positives against 71 false positives and 44 false negatives, 88.6% precision, 92.6% recall, 90.6% F1 overall.

This is not a small gap against the other methods, it is roughly an order of magnitude cleaner than AlignScore’s 23.6% strict precision or SummaC’s 12.5% and the reason is both identify as a root cause of those methods’ failures that NoteExtractor is not attempting cross document verification, so it never inherits the register mismatch between spoken transcript and written note that undermines every scorer in previous methods. It is answering an easier question, and its numbers reflect that rather than being a claim of a better method in the abstract.

9.6.3 Error Analysis & Failure Modes

NoteExtractor’s 71 false positives [Figure 14](#) carry a fundamentally different risk than a false positive from a verification checker like AlignScore or KDE. A verification checker’s false positive alarms the clinician about content that supposedly isn’t grounded in the transcript, exactly the alert fatigue failure mode

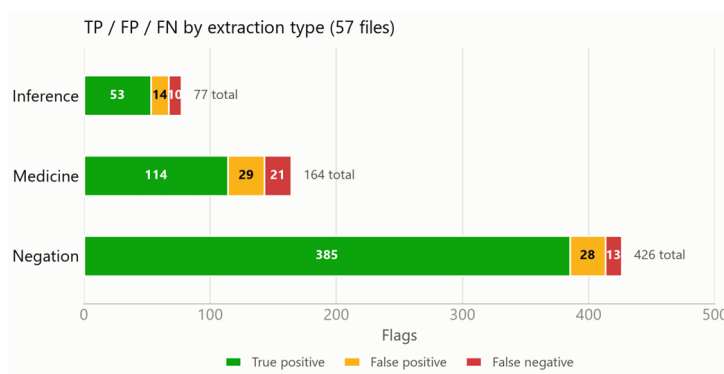


Figure 12: True positives, false positives, and false negatives by extraction type, across all 57 ground files. Negations are the most observed.

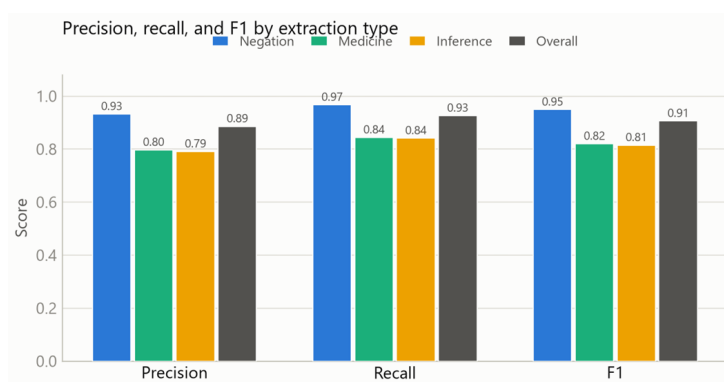


Figure 13: Precision, recall, and F1 by extraction type, with the overall combined score. Negation extraction (93%/97%/95%) is close to production-grade for this note style; medicine and inference sit in the high-70s/low-80s.



Figure 14: . Root cause of each of the 71 false positives. Roughly a third are labeling-set gaps rather than genuine extraction errors; the largest genuine mechanism (medicine ngram granularity, section-header carryover) mirrors failure modes already documented for AlignScore

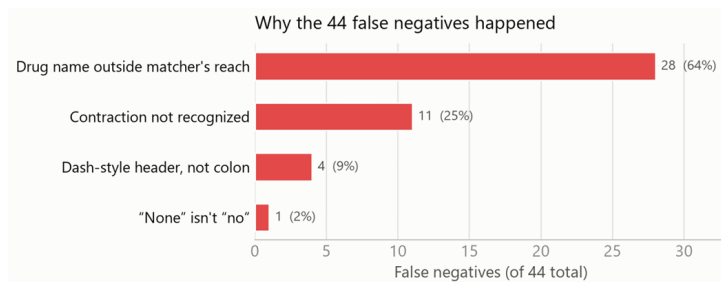


Figure 15: Root cause of each of the 44 false negatives. Nearly two-thirds trace to one mechanism: a drug or brand name outside the UMLS matcher’s near exact matching reach

that can turn a safety layer into a liability. NoteExtractor cannot make that mistake by construction: every flagged clause is a verbatim substring of the note the ambient AI wrote, not invented or pulled from elsewhere, so a “false positive” here means only that the checklist highlighted one more of the Ambient own sentences. It is another highlighted sentence to check, not a guarantee of a false claim.

The large majority of false positives are genuine medicine, negation, or inference mentions the checker caught correctly but the hand curated ground truth simply did not label “allergic to clindamycin” tagged as medicine, “does not always wear gloves” tagged as negation, “? need to confirm this” tagged as inference, read individually, none of these are wrong; they look more like gaps in labels_extraction’s own coverage than checker errors. Over inclusion in a same document extraction task like this one costs a clinician a moment’s glance at a sentence they should know the context of; under inclusion in a cross document verification task costs a missed hallucination or a false alarm on correct content. Which is why NoteExtractor’s precision can be read more forgivingly than the other methods in Table 5.

The false negatives in Figure 15 can be fixed by adjusting the NLP to using the bigger UMLS dataset and adjustment of the included negation words. Tuning is needed.

10 Results

Method	<i>n</i> files	Precision	Recall	F1	Avg. s/file
§9.2.2 medspaCy/UMLS/ConText checker	57	5.5%	16.7%	8.3%	12.35s
KDE (§9.3)	57	4.4%	44.7%	8%	4.23s
AlignScore, original (§9.4)	10	5.6%	5.9%	5.7%	44.4s
AlignScore, fixed pipeline, full 57-file [‡]	57	23.6%	41.2%	30.0%	124.4s
SummaC, original ZS (§9.5)	5	9.0%	50.0%	15.0%	1,828.9s
SummaC, GPU + fixes (§9.5)	1	12.5% [†]	100.0% [†]	22.2% [†]	191.6s
NoteExtractor (§9.6)	57	88.6%	92.6%	90.6%	0.24s

Table 5: Results of all the modules

Every verification method except NoteExtractor is, in its current state unusable: precision under 25% for six of eight rows above means a clinician using any of them would be shown a wrong flag roughly three times out of four or worse, the exact alert fatigue failure mode turns a safety layer into a liability.

The three independent mechanisms medspaCy, KDE and AlignScore, had one root cause: none of these methods, or the models/statistics underneath them, were built against spoken, disfluent, two-party clinical dialogue scored against a structured SOAP. Everyone of their data was validated, by its own authors, in its own published benchmark, against text that already looks like the other half of the comparison it was asked to perform. This is the problem this whole comparative evaluation surfaces: it is not that any one method is poorly built, or that any one threshold is miscalibrated; it is that verifying a spoken consultation against a written note is a harder, differently shaped problem than verifying one written document against another, and every general-purpose or written domain method suffers this

complication. NoteExtractor’s is the evidenced counterpoint to that pattern, not a contradiction of it: it surpasses specifically because it never has to solve the register mismatch problem. It answers a narrower question, ” what does this note contain?” entirely within the note’s own, written register. The register mismatch is a tax on cross document verification specifically; a method that only has to read one document does not pay it.

10.1 In Documents Tag Contribution

The performance disparity between cross-document verification tools (AlignScore, EmbedKDECheck, SummaC) and the in-document NoteExtractor (90.6% F1) exposes a blind spot in the current evaluation literature. Frameworks for factual consistency assume a uniform linguistic register between the source text and the generated summary, as seen in the benchmarking of both AlignScore and KDE on standardized medical reports. However, the empirical results of this study demonstrate that applying these state-of-the-art models to AVT introduces a ‘register mismatch.’ Spoken, disfluent, two-party clinical dialogue simply does not map cleanly to telegraphic, structured SOAP notes using current NLI or vector-density paradigms.

Therefore, this paper’s primary original contribution is a shift for clinical AI surveillance: moving away from flawed cross-document truth verification and toward in-document risk highlighting. By conceding the register mismatch and instead deploying a deterministic module (NoteExtractor) to isolate high-risk entities (medications, negations, and inferences) purely within the generated note, this architecture successfully bypasses the limitations that defeat current frontier models. In the broader context of AIaMD governance, this shifts the safety burden from algorithmic truth-checking—which remains computationally prohibitive and inaccurate—to risk-adapted cognitive friction. As a highly marketable, SaMD-compliant alternative to non-deterministic LLM judges, this targeted extraction method practically helps resolve the socio-technical risk of automation bias and provides the proactive, real-time telemetry pipeline necessary to modernize the MHRA’s retrospective Yellow Card scheme.

10.2 Cross-Document Verification Can Still Improve

The register mismatch diagnosis could be read as a case for abandoning deterministic verification checking altogether in favour of extraction only methods like NoteExtractor. AlignScore’s own re-evaluation history argues against reading it that pessimistically.

The implementation of AlignScore with a sentence splitter blind to clinical bullet-field punctuation, and a threshold calibrated on six hand-picked phrases rather than real note sentences were fixed directly [Figure 9](#) (Edit 1 and Edit 3), without touching the underlying RoBERTa alignment model. F1 moved from 0.077 to 0.423 on the original 10-file sample, a 5.5× improvement, with precision and recall both rising together rather than trading off.

Researching into the clinical dialogue disfluency can help the methods above become more reliable.

11 Example Market deployment

Five deterministic algorithms candidates were built and evaluated head to head, and NoteExtractor won decisively on the axis that matters for this paper’s stated architectural constraint an order of magnitude faster, and more precise, than any of the four verification checkers it was measured against. although there shows promise in some of the other methods, how to deploy these methods to benefit the yellow card scheme to adapt to AI safety is a critical factor that this paper will address.

This paper’s central claim is that the Yellow Card scheme leaves a safety vacuum because no one is watching continuously between deployment of Ambient AI models. A benchmarked but undeployed checker closes that vacuum no more than the scheme itself does. This section describes a working prototype that takes the current best method NoteExtractor out of the evaluation development environment and puts it into a clinician’s browser, against a real pseudo ambient AI scribe (DiSCaScribe) rather than only the offline prim57 corpus. DiSCaScribe is a mock Ambient AI used to test the DOM scraping-extension and provide a base for the proof of concept for the web extension.

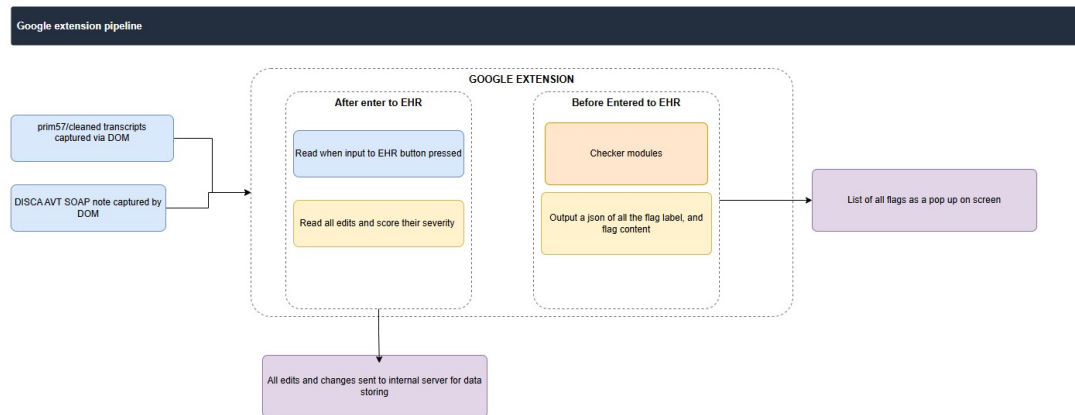


Figure 16: Google extension architecture.

11.1 What the Prototype Does

The prototype has two halves, running against the same captured note but answering two different questions. **A real time checklist.** While the note is still open, a floating widget reads the four SOAP fields directly out of the page and surfaces every medicine, negation, inference, and dose/frequency/duration/vital phrase NoteExtractor finds. Figure 17

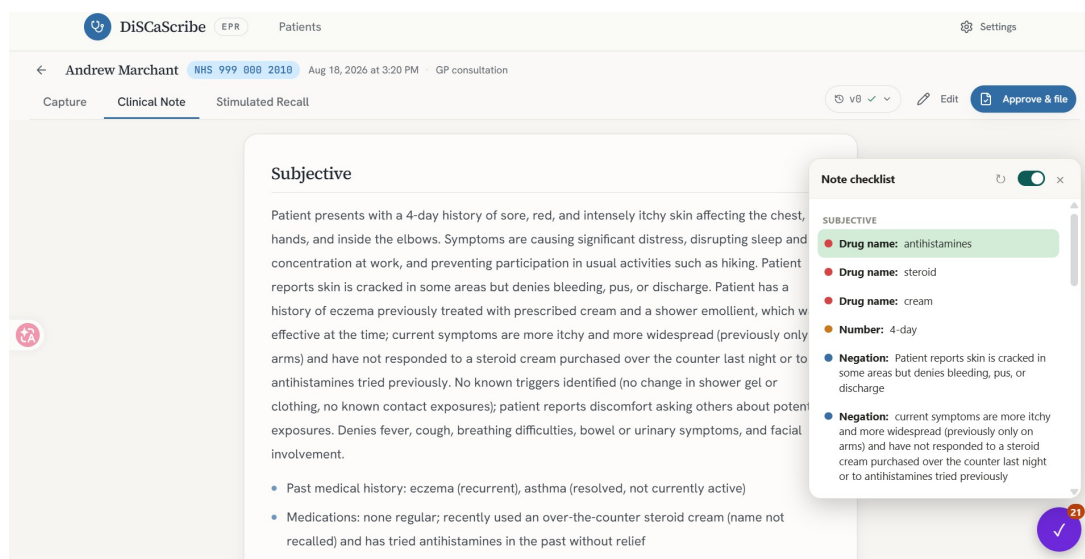


Figure 17: Preview of the google extension

Each item is colour-coded by type Figure 17, grouped by SOAP section, and clicking one scrolls to and highlights the matching text in the note itself. This is the checklist frontline clinicians need in place of the manual, hazard audit checklist Table 1 with the same review burden, but pre filtered to exactly the spans worth a second look, rather than requiring the clinician to re read the whole note field by field twice over.

By transforming the review process from a passive reading exercise into a targeted verification task, this interface directly attacks the psychological heuristic of automation bias. When frontline staff operate under severe time constraints, the cognitive path of least resistance is to implicitly trust the fluent, structured output of the AI, reducing human oversight to a repetitive, automated "click-through" loop (20). This passive acceptance not only allows safety-critical omissions to slip into the EHR but also accelerates long-term clinical deskilling. By actively isolating high-risk entities such as medications, inferences, and negations the widget forces the clinician into a state of active recall. Crucially, research by (55) demonstrates that visual cues do not need to be infallible to be effective; systems operating at just 75% reliability successfully improve target detection while preventing users from blindly trusting the

automation. Therefore, as long as a deterministic method proves to be at least 75% effective, it can be safely integrated into the extension to guide clinical attention. Given that the NoteExtractor module already operates at an overall F1 score of 90.6%, it comfortably exceeds this safety threshold. Introducing this deliberate, risk-adapted cognitive friction ensures the clinician remains actively engaged in the clinical reasoning loop. By enforcing this targeted attentiveness, the system disrupts the dangerous comfort of unsafe reassurance, optimizing the human-in-the-loop framework to catch the exact hallucinations and omissions that a fatigued, passive reader would otherwise miss.

A passive edit capture pipeline.

Separately from the checklist, the google extension snapshots the note the moment editing starts and again the moment the clinician saves, and ships both snapshots, plus the transcript, to a local server. Because this server infrastructure runs entirely on ordinary hospital network hardware without cloud dependency, it inherently mitigates data residency and GDPR exposure. No patient transcript or note ever leaves the local network to reach a third-party model API, ensuring strict privacy compliance while protecting the clinical pipeline from external data breaches. The server runs a purpose-built diff classifier that judges every changed line as critical or not using the same judgement as note extractor, and logs the result, timestamped, to an append-only CSV alongside a full JSON record of the session for auditing.

A dashboard aggregates this into visual analytics: a daily edit ratio trend line, a stacked breakdown of edit categories over time (drug switch, negation flip, number edit, inserted, omitted, reworded), and three mutually exclusive categories for the full session in buckets (no edit, edited-but-nothing-critical, and critical). This allows a safety officer to see at a glance whether a rising edit rate is stylistic noise or a rising share of genuinely dangerous corrections. This continuous, real-time monitoring paradigm mirrors modern national initiatives like the Federated AI Monitoring Service (FAMOS) (2), piloted at the Leeds Teaching Hospitals NHS Trust, which actively tracks model performance to catch calibration decay before patient harm occurs. Crucially, the system stores specific edits over sessions so that when incidents are reported to manufacturers, a definitive auditable trail is available [Figure 19](#). This digital retention directly addresses the structural failures of legacy PMS frameworks, such as the MHRA's Yellow Card Scheme and MORE. Because these legacy systems rely entirely on a voluntary, human-driven workflow, requiring an overworked clinician to manually identify and report an error from memory hours after an event, they are incapable of catching sub-visual semantic drift or probabilistic "silent failures". By maintaining an automated ledger of real-time corrections, the middleware empowers these reactive schemes; the exact anomalies are held by the software rather than relying on fallible human memory. Furthermore, tracking explicit human-AI interaction isolates genuine clinical corrections from algorithmic feedback loops, ensuring that automation bias does not cause clinicians to falsely validate flawed AI outputs, which would otherwise pollute the EHR.

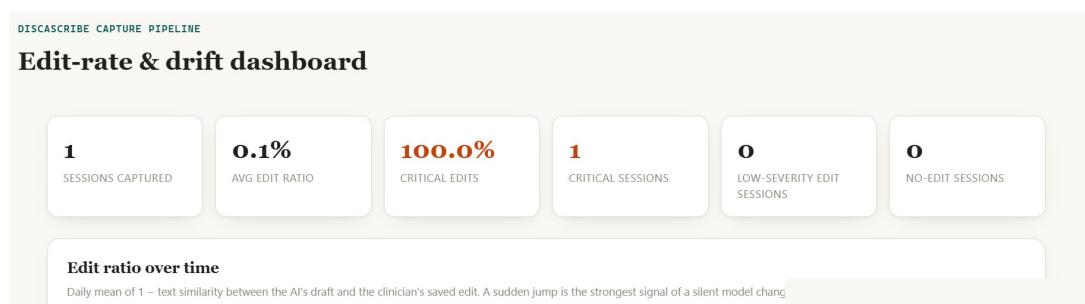


Figure 18: Dashboard stats example.

11.2 Why a Browser Extension Is the Right Delivery Mechanism

Modern AVT tools “do not deploy as standalone operating systems; instead, they function as an application layer integrated directly into existing clinical software frameworks,” (34) delivering their output “directly to the user interface via an API layer” that maps to enterprise EHR platforms the monitoring framework itself has no access to. This paper concludes from this that “a lightweight browser extension or desktop overlay can sit directly within the client side pipeline” by “reading the Document Object Model (DOM)...at the point of review.”

This section's prototype is the direct, working realisation of that conclusion. Three reasons this holds up in practice are:

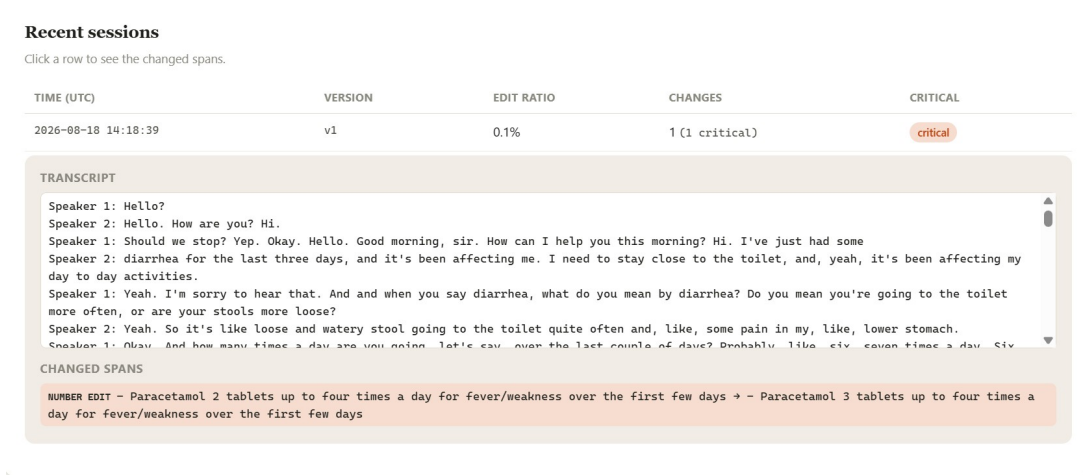


Figure 19: Dashboard recall example

Vendor independence: A browser extension observing rendered page content works against any AVT product with a web interface, without requiring manufacturer cooperation, a developer API grant, or trust in vendor reported self-testing.

Deployment friction low enough to actually pilot: A browser extension requires no installation to start capturing data. This matters directly against the critique of existing frameworks staying theoretical: a system that can be loaded unpacked into a browser and is already running end to end against a live AVT product's actual DOM structure is a materially different claim than a proposed architecture on paper.

Local-first, low-power infrastructure. The server component (Flask, CSV, no cloud dependency) runs on the same class of ordinary hospital network hardware and keeps every captured transcript and note on infrastructure the deploying institution controls rather than a third party API, the same locality argument makes for running AlignScore's checkpoint on device rather than calling a hosted model. This is not offered as a cost free architecture.

Reading a live DOM is inherently coupled to one vendor's markup at a point in time the exact selectors this prototype uses (soap-Subjective, the button titled "Save as the next note version", and so on) were obtained by direct inspection of DiSCaScribe's rendered page and will break silently if that vendor changes their UI. [Section 6.2](#) Many AVT manufacturers themselves frame DOM-reading as the fallback approach, preferable only when "seeking server side vendor API access" is not available, a developer API, where a vendor offers one, remains the architecturally cleaner option this prototype does not claim to replace. What a working extension demonstrates is that the fallback is viable now, today, against a real product, while a hypothetical universal API is not.

11.2.1 Closing the Loop

The Introduction's critique of the Yellow Card scheme is specific: it is voluntary, retrospective, and structurally dependent on a human first noticing that something has gone wrong before any report is ever filed.

published by greener et al.(28) , roughly 101,000 adverse drug reaction reports and 56,000 device incidents reach the MHRA annually, of which only "a microscopic fraction" is attributable to software or AI, not because AI related harm is rare but because the scheme was never built to surface silent, sub visual degradation of the kind [Section 3.1](#)describes. The Thalidomide era design this scheme still runs on assumes a patient or clinician can tell something is wrong (28) ; [Section 2.2](#) whole taxonomy of drift, omission, and hallucination risk is precisely the set of failure modes that assumption does not hold for. The edit capture pipeline re-frames reporting rather than merely automating it. It does not ask a clinician to notice anything, decide anything is worth reporting, or take any action beyond the one they were already going to take saving their note. Every save, made for whatever reason (a genuine correction, house style, tidying wording), becomes a timestamped, automatically classified data point, whether or not the clinician who made it ever consciously registers it as a "finding." This is the deployment level answer to what ("Modernizing Surveillance") and ("Continuous Real Time Monitoring") argue for only in the abstract: a dashboard that lets a safety officer see edit rate and edit category drift accumulate over time, without a single individual report ever needing to be filed for the signal to exist.

This closes a gap rather than replacing the scheme. A clinician who is actively alarmed by a dangerous AI output should, and still would, file a Yellow Card report nothing in this pipeline discourages that, and (“Modernizing Surveillance”) is explicit that the middle-ware layer is meant to empower the reactive scheme, not substitute for it, by ensuring historical evidence exists in software rather than relying on a clinician’s memory of an anomaly from hours prior.

What this closes is narrower and, more important: the cases nobody consciously notices at all, which is the failure mode [Section 3.1.2](#) highlights. Automation bias. Three limitations are worth stating, NoteExtractor’s own error sources, rather than glossed over.

First, the prototype has been built and smoke tested end to end against realistic sample notes and a real ambient scribe product’s DOM structure, but not yet run against a live clinical session.

Second, an edit based signal necessarily conflates a clinician’s personal editing style with a genuine AI error until the two are calibrated apart; the products own design notes already anticipate this and propose a lightweight, optional clinician quality signal (“this draft needed real fixes” vs. “just tidied wording”) as the calibration mechanism. Third, this remains a single-institution, single-vendor prototype; DOM-coupling caveat means porting to a second AVT product requires the same manual selector inspection process repeated, not a configuration change.

12 Method Limitations

While the deterministic methods explored in this work succeed in providing auditable, reproducible guardrails, behaving in alignment with standard SaMD regulations rather than AIaMD, they introduce their own technical constraints:

Symbolic NLP Brittleness: Rule based systems like medspaCy rely heavily on explicitly defined vocabularies (e.g., UMLS) and hardcoded syntactic rules. Consequently, these methods require constant manual updates to accommodate new clinical shorthand, evolving medication names, or novel diseases.

Embedding Space Mismatches: Statistical density checks (EmbedKDECheck) assume the underlying vector space accurately represents the target domain. Relying on generic English embeddings, as done for this paper’s implementation, (Word2Vec/FastText) rather than vectors fine-tuned explicitly on spoken UK primary care dialogue creates severe out-of-vocabulary blind spots and density inaccuracies. Updating these vector spaces as new medical areas grow is key, to ensure there are not constant blindspots after deployment.

Static Model Decay: trained deterministic models (AlignScore) are frozen at their training checkpoint. While this fixed inference guarantees reproducibility (unlike generative AI), it leaves the verification model vulnerable to data drift over time if the clinical register evolves. So the model will have to be updated over time.

Hardware and Computational Ceilings: Matrix heavy NLI approaches like SummaC were completely disqualified due to their computational complexity; requiring up to three minutes of processing per file on a high spec commercial system, renders them fundamentally incompatible with the real-time, CPU-bound hardware constraints of standard NHS trusts.

DOM-Coupling: The proposed client side browser extension relies on reading the live Document Object Model (DOM). This creates a brittle infrastructure that will silently break if the AVT vendor updates their user interface markup without warning.

Despite the continuous maintenance overhead required to update these deterministic modules, whether appending new UMLS concepts, refining NLP syntactic rules, or retraining static alignment models, this architectural rigidity is what makes them a safer monitoring layer than the AI they evaluate. Generative AIaMD operates as a probabilistic “black box,” where the underlying weight distributions and logic behind a hallucinated fact or an omitted clinical detail cannot be fully audited or explained. In contrast, deterministic methods guarantee a transparent, auditable trail. If a rule-based checker misses a newly introduced pharmaceutical, or a density threshold misflags a sentence, the exact point of algorithmic failure can be explicitly traced, logged, and corrected without the risk of unpredictable downstream side effects. By adhering to traditional SaMD principles, this non-black-box architecture ensures that the safety net itself does not introduce secondary probabilistic risks into the clinical pipeline, making the manual update burden a highly acceptable trade-off for strict regulatory accountability and patient safety.

13 Conclusion

The integration of AIaMD offers proven potential to alleviate clinical administrative burdens. However, the current regulatory landscape relies heavily on legacy PMS tools such as the Yellow Card Scheme and the MORE, which are retrospective and reliant on voluntary human reporting. Because these frameworks require clinicians to manually identify errors hours after an event, they suffer a blind spot regarding probabilistic AI risks, remaining structurally incapable of catching “silent failures,” subtle semantic drift, or the consequences of automation bias. To close this safety vacuum, this paper explored the deployment of deterministic algorithmic checkers designed to provide auditable, real-time safety guardrails. The research tracked a technical trajectory to bridge the gap between generated SOAP notes and raw transcripts;

- Initial rule-based NLP (medspaCy) struggled severely with the register mismatch between spoken clinical dialogue and written summaries.
- EmbedKDECheck attempted to bypass rigid phrasing rules by measuring semantic density via PCA, but failed to distinguish dropped clinical facts from clinically irrelevant conversational filler.
- AlignScore introduced a trained alignment model to score factual consistency directly, improving precision significantly (reaching an F1 of 0.423 after calibration), but still struggled with implicit dialogue Q&A bridging.
- SummaC was tested to handle document-level alignment but was disqualified entirely by prohibitive computational runtimes. Ultimately, the most viable solution emerged from narrowing the scope.
- The NoteExtractor module successfully bypassed the cross-document register mismatch entirely by answering an in-document question (“what does this note contain?”), achieving a 90.6% F1 score.

By operationalizing NoteExtractor into a live browser extension and local API, this work demonstrates that passive monitoring is technically viable today. This Proof of Concept mitigates automation bias by actively isolating high-risk clinical entities for review, while simultaneously capturing an auditable trail of telemetry edits to track long-term model drift without adding overhead to the clinician.

14 Legal, Social, Ethical, and Professional Issues (LSEPI)

14.1 Ethical Review and Data Governance

To avoid the severe data breach risks associated with live patient identifiable data, this project deliberately utilised the PriMock57 dataset. As this corpus consists of synthetic, mocked consultations conducted by actors and clinicians, the research aligns with GDPR and the UK Caldicott Principles. This choice bypasses the need to handle sensitive medical records, satisfying standard data governance requirements and exempting the project from requiring formal Imperial College Research Ethics Committee (ICLREC) approval.

14.2 Patient Consent and Acoustic Surveillance

The deployment of Ambient Voice Technology (AVT) in general practice introduces acoustic surveillance concerns. Ethical deployment mandates strict consent protocols, including opt-in verbal consent and clear visual recording indicators in the clinic. Furthermore, operational systems must be designed to securely handle or discard incidental third-party disclosures and confidential bystander data captured during the consultation. This paper uses a mock AVT which simulates an AVT but is not in use of any institution as in a development play area. As well as using the Primock57 datasets audio database for simulated input for the AVT, no consent is needed as it is a publicly available simulated dataset.

14.3 Medico-Legal Liability and Duty of Care

When an AIaMD system hallucinates a medication dosage or omits a critical red-flag symptom, the final medico-legal liability remains a complex ethical issue, whether the on the manufacturer or the clinician using the product. Under the UK Medical Devices Regulations 2002 (UK MDR 2002) (56) and General Medical Council (GMC) (57) prescribing guidance, the duty of care and legal accountability reside strictly with the clinician who signs off on the generated SOAP note. This underscores the professional necessity of The Sentinel System to prevent automation bias and protect clinical liability.

14.4 Health Equity and Algorithmic Bias

A critical social issue in ambient clinical AI is algorithmic bias and its impact on health equity. Probabilistic transcription models are highly susceptible to covariate shift [Section 3.1.1](#) when processing regional dialects (such as Scottish, Welsh, or multicultural London English), non-native accents, and specific pediatric or geriatric speech patterns. This demographic performance degradation risks systemic transcription inaccuracies for protected groups, making continuous real-time monitoring essential to ensure equitable care delivery. The sentinel systems recording pipeline can record the major edits of the SOAP notes and can identify which notes are being edited the most, whether that be from a specific accent or other inequalities. Which can then be reported by the yellow card scheme as well as be a record of proof for the MORE scheme.

15 Policy Recommendation: A White Paper for AIaMD Governance

The findings of this paper demonstrate that while AIaMD introduces efficiency into clinical workflows, its probabilistic nature fundamentally outpaces existing Post-Market Surveillance infrastructure.

Therefore, this paper formally proposes a modernization of the MHRA's Yellow Card Scheme to bridge the gap between probabilistic AI and the auditable safety standards of traditional Software as a Medical Device. To achieve this, this thesis proposes three core regulatory mandates for healthcare AI governance:

- **Automating the Yellow Card Scheme via Passive Telemetry:** This paper proposes that regulators mandate AIaMD deployments be coupled with independent, client-side middleware that passively logs edit distances and human-AI interaction times. By funnelling this telemetry into an auditable dashboard, the reporting is shifted from the clinician's fallible memory to a deterministic software ledger.
- **Judging AI through SaMD Style Deterministic Auditing:** Non-deterministic AIaMD must be continuously evaluated by transparent, non-black-box algorithms (such as the proven NoteExtractor module). This paper argues that enforcing this ensures oversight mechanisms provide a clear, reproducible, and auditable trail of anomalies.
- **Mandating UI guardrails to Combat Automation Bias:** Finally, this paper proposes updating clinical safety standards to legally require the integration of deterministic cognitive aids. Mandating interfaces that actively highlight high-risk entities introduces necessary cognitive friction, forcing active recall and dismantling the dangerous "click-through" verification loop.

16 Future Work

While the NoteExtractor POC successfully demonstrates the viability of passive monitoring, future technical development is required to address the architectural limitations identified in this study.

Future iterations of the cross-document verification models (such as AlignScore and EmbedKDECheck) must be fine-tuned explicitly on UK primary care transcripts rather than generic web text, allowing the algorithms to accurately bridge the gap between spoken disfluencies and structured clinical summaries.

The current prototype's reliance on client-side DOM reading creates a brittle infrastructure. Future work should focus on developing standardized, vendor-agnostic APIs that allow hospital telemetry dashboards to interface directly and securely with AI scribe software, removing the vulnerability of sudden user interface updates.

References

- [1] Junaid Bajwa, Usman Munir, Aditya Nori, and Bryan Williams. Artificial intelligence in healthcare: transforming the practice of medicine. *Future Healthcare Journal*, 8(2):e188–e194, Jul 2021. doi: 10.7861/fhj.2021-0095.
- [2] Medicines & Healthcare products Regulatory Agency (MHRA). Ai airlock sandbox pilot programme report. Technical report, Medicines and Healthcare products Regulatory Agency, London, 2025.
- [3] The Royal College of Radiologists (RCR). Post-deployment monitoring and safety reporting of ai medical imaging devices in clinical practice. Technical report, The Royal College of Radiologists, London, 2026.
- [4] IMDRF SaMD Working Group. Software as a medical device (samd): Key definitions. Technical report, International Medical Device Regulators Forum, 2013.
- [5] Medicines and Healthcare products Regulatory Agency (MHRA). Software and ai as a medical device change programme. Technical report, Medicines and Healthcare products Regulatory Agency, London, 2021.
- [6] S. Kothari, P. Kumar, M. Harrison, and R. Patel. System degradation and performance drift in production machine learning: Methods for passive detection and temporal distribution alignment. *IEEE Transactions on Knowledge and Data Engineering*, 31(8):1542–1555, 2019. doi: 10.1109/TKDE.2019.2891104.
- [7] H. Guan, D. Bates, and L. Zhou. Keeping medical ai healthy and trustworthy: A review of detection and correction methods for system degradation. *IEEE Transactions on Biomedical Engineering*, pages 1–38, 2026. Available at: PMC 2026 April 05.
- [8] A. Oukelmoun, N. Semmar, G. de Chalendar, C. Cormi, M. Oukelmoun, E. Vibert, and M. A. Allard. Detecting omissions in llm-generated medical summaries. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing: Industry Track*, pages 325–337. Association for Computational Linguistics, 2025.
- [9] M. B. Aithal, A. Kotz, and J. Mitchell. Placid: Privacy-preserving large language models for acronym clinical inference and disambiguation. *arXiv preprint*, arXiv:2603.23678v1 [cs.CL]:1–12, 2026.
- [10] C. Van Oyen and N. Mirza-Haq. Medsafe-dx (v0): A safety-focused benchmark for evaluating llms in clinical diagnostic decision support. *medRxiv preprint*, 2026. doi: 10.64898/2026.04.14.26350711.
- [11] U.S. Food and Drug Administration, Health Canada, and Medicines and Healthcare products Regulatory Agency (MHRA). Good machine learning practice for medical device development: Guiding principles. Technical report, U.S. Food and Drug Administration, Oct 2021. URL <https://www.fda.gov/media/153486/download>.
- [12] IntuitionLabs. Fda pccp guide for ai/ml software as a medical device, Apr 2026. URL <https://intuitionlabs.ai/articles/fda-pccp-implementation-guide-ai-ml-samd>. Accessed: 2026-07-11.
- [13] Vention. Ai in UK healthcare: Statistics and trends, 2026. URL <https://ventionteams.com/uk/healthcare/ai/statistics-and-trends>. Accessed: 2026-07-11.
- [14] Microsoft Source. Nhs england accelerates ai adoption with microsoft 365 copilot to improve service delivery, reduce costs and create more time for care, Jun 2026. URL <https://news.microsoft.com/source/2026/06/08/nhs-england-accelerates-ai-adoption-with-microsoft-365-copilot-to-improve-service-delivery-redu>. Accessed: 2026-07-11.
- [15] Samuel G. Finlayson, John D. Bowers, Joichi Ito, Jonathan L. Zittrain, Andrew L. Beam, and Isaac S. Kohane. Adversarial attacks on medical machine learning. *Science*, 363(6433):1287–1289, 2019. doi: 10.1126/science.aaw4399.

- [16] Atomic Spin. Your coding agents are only as good as their feedback loop, Jul 2026. URL <https://spin.atomicobject.com/recent-articles/>. Accessed: 2026-07-11.
- [17] Evidently AI. Monitoring data drift in ml models: Workflows and tools, 2024. URL <https://www.evidentlyai.com/blog/data-drift>. Accessed: 2026-07-11.
- [18] Chuan Guo, Geoff Pleiss, Yu Sun, and Kilian Q. Weinberger. On calibration of modern neural networks. In *Proceedings of the 34th International Conference on Machine Learning (ICML)*, pages 1321–1330. PMLR, 2017.
- [19] Kate Goddard, Abdul Roudsari, and Jeremy C Wyatt. Automation bias: a systematic review of frequency, effect mediators, and mitigators. *Journal of the American Medical Informatics Association*, 19:121–127, 2012. doi: 10.1136/amiajnl-2011-000089.
- [20] Raja Parasuraman and Dietrich H. Manzey. Complacency and bias in human use of automation: An attentional integration. *Human Factors*, 52(3):381–410, 2010. doi: 10.1177/0018720810376055.
- [21] M. Lyu, X. Li, Z. Chen, J. Pan, C. Peng, S. Talankar, and Y. Wu. Natural language generation in healthcare: A review of methods and applications. *Journal of Biomedical Informatics*, 176(104997): 1–27, 2026.
- [22] Roxana Daneshjou, Kailas Vodrahalli, Roberto A. Novoa, Margaret Jenkins, Weixin Liang, Veronica Rotemberg, Allan Halpern, and James Zou. Disparities in dermatology ai performance on a diverse, curated clinical image set. *Science Advances*, 8(32):eabq6147, 2022. doi: 10.1126/sciadv.abq6147.
- [23] K. N. Vokinger et al. Continual learning in medical devices: FDA’s action plan and beyond. *The Lancet Digital Health*, 3(6):e337–e338, 2021. PubMed: 33933404.
- [24] U.S. Food and Drug Administration. Artificial intelligence and machine learning in software as a medical device, 2019. URL <https://www.fda.gov/medical-devices/software-medical-device-samd/artificial-intelligence-and-machine-learning-software-medical-device>. Accessed: 4 July 2022.
- [25] J. H. Chen et al. Decaying relevance of clinical data towards future decisions in data-driven inpatient clinical order sets. *International Journal of Medical Informatics*, 102:71–79, 2017. PubMed: 28495350.
- [26] Z. Young and R. Steele. Empirical evaluation of performance degradation of machine learning-based predictive models—A case study in healthcare information systems. *International Journal of Information Management Data Insights*, 2(1):100070, 2022.
- [27] N. Vargesson. Thalidomide-induced teratogenesis: History and mechanisms of action. *BioMed Research International*, 2015:1–14, 2015. doi: 10.1155/2015/241016.
- [28] Robert Greener, Alan Jones, and Sarah Tomkins. Integrating alternative products into national safety frameworks: An evaluation of e-cigarette adverse incident reporting under the yellow card scheme. *International Journal of Drug Policy*, 43(2):88–95, 2017. doi: 10.1016/j.drugpo.2017.02.011.
- [29] British Society of Interventional Radiology (BSIR). Standards and resource assessment for medical device safety reviews in nhs trusts. BSIR Professional Technical Report, 2026.
- [30] Newton’s Tree and Leeds Teaching Hospitals NHS Trust. Federated ai monitoring service (famos): Proactive ai monitoring and deployment in healthcare. GOV.UK AI Assurance Case Study and Technical Report, 2026. Supported by Innovate UK.
- [31] Kate Goddard, Abdul Roudsari, and Jeremy C. Wyatt. Automation bias – a hidden issue for clinical decision support system use. In *International Perspectives in Health Informatics*, volume 164 of *Studies in Health Technology and Informatics*, pages 17–22. IOS Press, 2011. doi: 10.3233/978-1-60750-709-3-17.

- [32] HIT Consultant. Beyond the hype: New KLAS data validates the financial and clinical ROI of ambient AI, jan 2026. URL <https://hitconsultant.net/2026/01/21/klas-roi-validation-2026-ambient-ai-results/>. Accessed: 2026-07-15.
- [33] NHS England Digital. Ambient voice technology self-certified supplier registry, jun 2026. URL <https://digital.nhs.uk/services/ambient-scribing/ambient-voice-technology-self-certified-supplier-registry>. Last edited: June 23, 2026. Accessed: 2026-07-15.
- [34] TORTUS AI. TORTUS: The ambient AI scribe for healthcare, 2026. URL <https://tortus.ai/>. Accessed: 2026-07-15.
- [35] Heidi Health. Safety at heidi, 2026. URL <https://www.heidihealth.com/safety>. Accessed: 2026-07-15.
- [36] Suki AI. What is ambient clinical intelligence? the 2026 guide for health systems, 2026. URL <https://www.suki.ai/blog/what-is-ambient-clinical-intelligence-the-2026-guide-for-health-systems/>. Accessed: 2026-07-15.
- [37] Alex Papadopoulos Korfiatis, Francesco Moramarco, Radmila Sarac, and Aleksandar Savkov. Pri-Mock57: A dataset of primary care mock consultations. *arXiv preprint arXiv:2204.00333*, 2022. doi: 10.48550/arXiv.2204.00333. URL <https://arxiv.org/abs/2204.00333>.
- [38] Scott M. Lundberg and Su-In Lee. A unified approach to interpreting model predictions. In *Advances in Neural Information Processing Systems 30 (NeurIPS 2017)*, 2017. URL <https://arxiv.org/abs/1705.07874>.
- [39] Marco Tulio Ribeiro, Sameer Singh, and Carlos Guestrin. "why should i trust you?": Explaining the predictions of any classifier. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 2016. URL <https://arxiv.org/abs/1602.04938>.
- [40] Bhavik Vachhani, Kush Shrivastava, Pranshu Nema, and Sai Chiranthan. Beyond literal summarization: Redefining hallucination for medical soap note evaluation, 2026. URL <https://arxiv.org/abs/2604.14829>.
- [41] Kiril Gashteovski, Rainer Gemulla, and Luciano Del Corro. Minie: Minimizing facts in open information extraction. *Proceedings of the 2017 Conference on Empirical Methods in Natural Language Processing*, n.d.n.d., 2017. doi: 10.18653/v1/d17-1278.
- [42] Hannah Eyre, Alec B. Chapman, Kelly S. Peterson, Jianlin Shi, Patrick R. Alba, Makoto M. Jones, Thomas L. Box, Scott L. DuVall, and Oliver V. Patterson. Launching into clinical space with medspaCy: a new clinical text processing toolkit in Python. *arXiv preprint arXiv:2106.07799*, 2021. doi: 10.48550/arXiv.2106.07799. URL <https://arxiv.org/abs/2106.07799>.
- [43] Matthew Honnibal and Ines Montani. spaCy 2: Natural language understanding with Bloom embeddings, convolutional neural networks and incremental parsing. <https://spacy.io>, 2017.
- [44] Steven Bird, Ewan Klein, and Edward Loper. *Natural Language Processing with Python*. O'Reilly Media, 1st edition, 2009. ISBN 978-0596516499.
- [45] Mark Neumann, Daniel King, Iz Beltagy, and Waleed Ammar. ScispaCy: Fast and robust models for biomedical natural language processing. In *Proceedings of the 18th BioNLP Workshop and Shared Task*, pages 319–327. Association for Computational Linguistics, 2019. doi: 10.18653/v1/W19-5034. URL <https://aclanthology.org/W19-5034>.
- [46] J. Pizarro. negspacy: spacy pipeline object for negating concepts in text, n.d. URL <https://github.com/jenojp/negspacy>.
- [47] Wendy W. Chapman, William Bridewell, Paul Hanbury, Gregory F. Cooper, and Bruce G. Buchanan. A simple algorithm for identifying negated findings and diseases in discharge summaries. *Journal of Biomedical Informatics*, 34(5):301–310, 2001.

- [48] Henk Harkema, John N. Dowling, Tyler Thornblade, and Wendy W. Chapman. Context: An algorithm for determining negation, experiencer, and temporal status from clinical reports. *Journal of Biomedical Informatics*, 42(5):839–851, 2009.
- [49] Olivier Bodenreider. The unified medical language system (umls): integrating biomedical terminology. *Nucleic Acids Research*, 32(Database issue):D267–D270, 2004.
- [50] Alan R. Aronson. Effective mapping of biomedical text to the umls metathesaurus: The metamap program. In *Proceedings of the AMIA Symposium*, pages 17–21, 2001.
- [51] Luca Soldaini and Nazli Goharian. Quickumls: a fast, unsupervised approach for medical concept extraction. In *Proceedings of the Medical Information Retrieval (MedIR) Workshop at SIGIR*, 2016.
- [52] Naoaki Okazaki and Jun’ichi Tsujii. Simple and efficient algorithm for approximate dictionary matching. In *Proceedings of the 23rd International Conference on Computational Linguistics (COLING 2010)*, pages 851–859, 2010.
- [53] Y. Zha, Y. Yang, R. Li, and Z. Hu. Alignscore: Evaluating factual consistency with a unified alignment function. *arXiv preprint arXiv:2305.16739*, 2023. Presented at ACL 2023.
- [54] Philippe Laban, Tobias Schnabel, Paul N. Bennett, and Marti A. Hearst. Summac: Re-visiting nli-based models for inconsistency detection in summarization. *Transactions of the Association for Computational Linguistics*, 10:163–177, 2022. doi: 10.1162/tacl_a.00453.
- [55] Michelle Yeh and Christopher D. Wickens. Display signaling in augmented reality: Effects of cue reliability and image realism on attention allocation and trust calibration. *Human Factors*, 43(3): 355–365, 2001. doi: 10.1518/001872001775898269. URL <https://journals.sagepub.com/doi/10.1518/001872001775898269>.
- [56] The medical devices regulations 2002, 2002. URL <https://www.legislation.gov.uk/uksi/2002/618/contents>.
- [57] General Medical Council. Good medical practice, 2024. URL <https://www.gmc-uk.org/professional-standards/the-professional-standards/good-medical-practice>. Accessed: 2026-08-27.

17 Appendix A: Code

This appendix outlines the directory tree of the projects codebase as well as explaining the structure of the code and its dependencies. The repo for this project is public and can be found at: https://github.com/NDimish/imperial_thesis

17.1 Directory tree

```
project/
|-- run_checker_modules.py      (run all checker modules with all labels/transcript/soaps)
|-- run_condensers.py          (run the condensor test to choose best nlp)
|-- run_note_extractor.py      (Run not extractor pipeline)
|-- run_single_pair.py         (runs all modules for 1 single transcript/label/soap)
|-- umls_checker.py            (checking the UMLS denylist and words work)
|-- gemini_rate_limiter.py     (Rate limmier for using gemini so not to go over rate limit)
|-- secrets_config.py          (Contains secret api codes for google extension and any AI checkers)
|-- requirements.txt
|
|-- batch_runs/ (running specific files per module)
|   |-- ai_checker_batch.py
|   |-- stage2_judge_batch.py
|   |-- kde_batch57.py
|   |-- medspacy_umls_batch57.py
|   '-- summac_batch.py
|
|-- Modules/
|   |-- __init__.py
|   |-- base.py                (CheckerModule class)
|   |-- evaluate.py            (Evaluate class for seeing if values correct)
|   |-- risk_taxonomy.py       (idetifying the risk level of the flag)
|   |-- note_extractor.py
|   |-- edit_diff_checker.py   ( difference between two files used for edit chcking in extension)
|   |-- AI_checker.py          (AIChecker)
|   |-- alignscorechecker2.py  (AlignScoreChecker)
|   |-- embedkde_checker.py    (EmbedKdeChecker)
|   |-- medspacy_umls_checker.py (MedspacyUmlsChecker)
|   |-- metamap_cui_checker.py (MetaMapCuiChecker, MetaMapLiteClient)
|   |-- summac_checker.py      (SummaCChecker)
|
|-- Medical condensor/
|   |-- base.py                (CondenserModule)
|   |-- medspacy_condenser_new.py (MedspacyCondenserNew)
|   |-- quickumls_condenser.py  (QuickUMLSCondenser)
|   |-- kdbe_check.py
|   |-- umls_matching.py       (The umls used for the modules)
|   |-- negspacy_condenser.py   (negspaCy condensor)
|   |-- scispacy_condenser.py   (scispaCy condensor)
|   |-- evaluate.py
|
|-- datamakerfiles/
|   |-- combine_audio.py
|   |-- prim57_cleaner.py
|   |-- soap_maker.py
|   |-- prim_lib_injection.py   (makes bad labels, hallucination and ommisions)
|   '-- prim_lib_injection_extra.py (Adds more detailed labels with LASA and number changes)
|
|-- slurm/
```

```
| |-- sbatch_var.sh
| |-- run_main.sh
| |-- run_nlp_main.sh
| '-- run_rebuttal_tests.sh    (files to run system on slurm (High performace compute))
|
|-- prim57/
    (working dataset: transcripts, notes, labels, for bothSOAP checker and note extractor)
|-- Clean Transcripts/      (larger raw source transcript pool)
|-- Logs/                   (per-run precision/recall log+json, plots)
'-- google api/             (browser extension code)
```

17.2 Files under Project

The files under the main directory are the files that import the modules and evaluators so to run each module over the dataset then create a log explaining what was caught and missed as well as calculating F1,precision,accuracy scores.

17.2.1 run_checker_modules.py

Runs the configured checker modules over every prim57 transcript/SOAP-note pair, condensing each transcript first, scoring against ground-truth labels via **Evaluate**.

- First function load_checker_modules() – imports and instantiates each SOAP checker; If the checker is disables it skips (and reports if skipped)
- Second functions are read_input_file(filename, condenser), read_output_file(filename). These functions reads the files formatted in txt and makes them in a binary format that the code can read.
- The main function main(limit=None) is the run loop: condense, check, Evaluate.compare() per file then runs Evaluate.results() per module

17.2.2 run_condensers.py

Runs each NLP condenser over transcript/ground-SOAP pairs, scores condensing quality, and plots per-run trends.

- First function run load_condenser_modules() – same skip pattern as above
- Second read_file(directory, filename). Reads the transcripts then converts to code readable format
- Third run is plot_trends(all_checkpoints) – per-run trend plot (elapsed time, word re-duction, KDE/cosine/ROUGE omission & groundedness diffs)

17.2.3 run_note_extractor.py

Runs **NoteExtractor** over prim57's real SOAP notes, scored against **prim57/labels_extraction**.

- Function run is main(limit=None) – the only function in this file; calls Modules.note_extractor.extract_note_content module directly without need for loader and then runs similar to run_checker_modules.py.

17.3 Environment setup

1. **Python env.** Any Python 3.11 conda env or venv works (the cluster scripts for SLURM (High performance compute running) default to a conda env named soap-checker.
2. **Install requirements:**

```
pip install -r requirements.txt
```
3. **Three packages need installing separately**, bypassing stale upstream version constraints (see the comment block at the top of **requirements.txt**):

```

pip install --no-deps medspacy==1.3.1 PyRuSH==1.0.13
pip install --no-deps \
  git+https://github.com/yuh-zha/AlignScore.git@a0936d5afee\
  642a46b22f6c02a163478447aa493

```

4. **Biomedical spaCy model:** `en_core_sci_sm` is already pinned as a direct URL in `requirements.txt`, so step 2 installs it; no separate download needed.
5. **QuickUMLS** (needed by most checkers/condensers – the shared concept matcher in Medical condensor/`umls_matching.py`). Install the `quickumls` package and build/download a local UMLS installation separately (outside this repo), then point the code at it: edit `QUICKUMLS_INSTALL_DIR` near the top of `Medical condensor/umls_matching.py` to your install path :it is a hardcoded constant, not an environment variable, so this file must be edited on every new machine. The UMLS database will need clearance to download. Requesting for clearance is free.
6. **GPU.** `SummaCChecker` defaults to `device="cuda"` : either run it on a CUDA-capable machine or pass `device="cpu"` yourself before running it.
7. **Output location (optional).** Every script writes its logs/JSON under **Logs/** by default; override with:

```
export RESULTS_DIR=/path/to/results
```

17.4 Running things

17.4.1 Full-dataset evaluation runs

The `limit` variable indicates how many files to be run. Where the files to read and compare against are already hard coded into the python file. if `limit` is 5, 5 random files will run from the directory of the dataset. Max is 57.

- **`python run_condensers.py [limit]`** runs every condenser that still loads (**`SciSpacyCondenser`**, **`NegspacyCondenser`**, **`QuickUMLSCondenser`**; **`MedspacyCondenser`**) over transcript/ground-SOAP pairs, plots trends to **`Logs/nlp_main_trends.png`**.
- **`python run_note_extractor.py [limit]`**: runs **`NoteExtractor`** over `prim57`'s real SOAP notes, scored against **`prim57/labels_extraction`**.
- **`python run_checker_modules.py [limit]`** : runs every checker module that still loads (**`medspaCY`** and **`umls CUI`**, **`embedKDE`**, **`Alignscore`**; **`Summac`**) over transcript/ground-SOAP pairs, plots trends to **`Logs/nlp_main_trends.png`**.

18 Appendix B : Running the google extension

Two independent pieces, all living under google api/:

- **extension:** a Manifest V3 Chrome extension that runs only on https://discascribe.vercel.app/consultations/*. It reads the Clinical Note tab's four SOAP fields (Subjective/Objective/Assessment/Plan) and the Capture tab's transcript, asks the local server what is worth double-checking, and renders the answer as a floating checklist widget. It also snapshots every edit pass (the note when Edit mode is entered vs. the note when Save is clicked) and ships both versions plus the transcript to the server.
- **server:** a local Flask app (server.py) that runs the checklist extraction and the before/after diff, logs every edit pass to a timestamped CSV, and serves a dashboard at /dashboard. .

The connection between the server and extension is through the computer own network. Later configured for own hospital server networks. So this can work in LAN. (localhost:5000) and the DiSCaScribe page (Or any other AVT configured) itself are the two networks needed. The logic that decides what to flag or classify is shared, plain-Python code in Modules/ at the project root, note_extractor.py and edit_diff_checker.py, so it has no dependency on Flask or the extension and can be unit-tested from the command line.

18.1 Prerequisites

- **Python environment.** The medspaCy/QuickUMLS stack this project uses lives in a conda environment with imports from requirements.txt. That is what server.py needs on its import path, it pulls in Modules/note_extractor.py and Modules/edit_diff_checker.py, both of which load the shared QuickUMLS matcher.
- **Flask.** Installed into conda environment already as part of building this; if starting from a fresh environment, pip install requirement.txt at the project root).
- **Google Chrome** (or another Chromium-based browser) with developer mode available, this is a Manifest V3 unpacked extension, not something installed from the Chrome Web Store.

18.2 Running the server

From the project root, with the conda environment active:

```
conda activate (ENV name)
python "google api/server/server.py"
```

This starts the server on <http://localhost:5000> and serves the dashboard at <http://localhost:5000/dashboard>. Leave it running while using the extension. **Run the server first** then the extension otherwise the extension has nothing to talk to without it, and the widget will show *"Can't reach the local server"* if it is down.

Data is written to google api/server/data/ (gitignored):

- events.csv: one row per edit pass, the durable timestamped.
- sessions/*.json: one file per edit pass with the full transcript/note/diff behind that row, for the dashboard's specifics area

18.3 Loading the extension into Chrome

1. Open chrome://extensions in Chrome.
2. Turn on **Developer mode** (top-right toggle).
3. Click **Load unpacked** and select the google api/extension/ folder.
4. Navigate to <https://discascribe.vercel.app/consultations/...> a small purple circle should appear bottom right of the page once the server is also running. like in Figure 20

Click the extension's toolbar icon at any time to flip the master on/off switch, or to jump straight to the dashboard.

EDIT

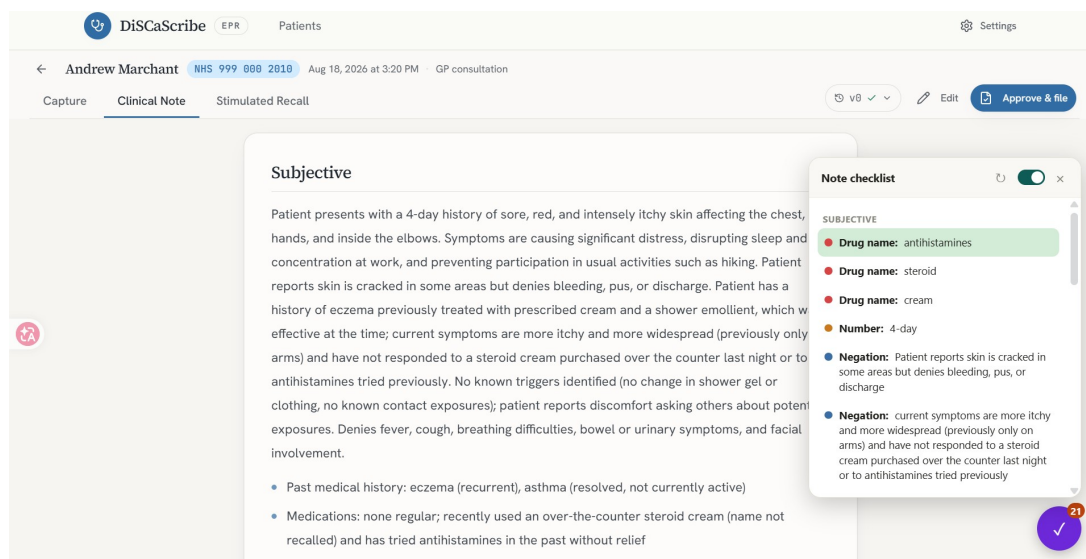


Figure 20: Preview of the google extension

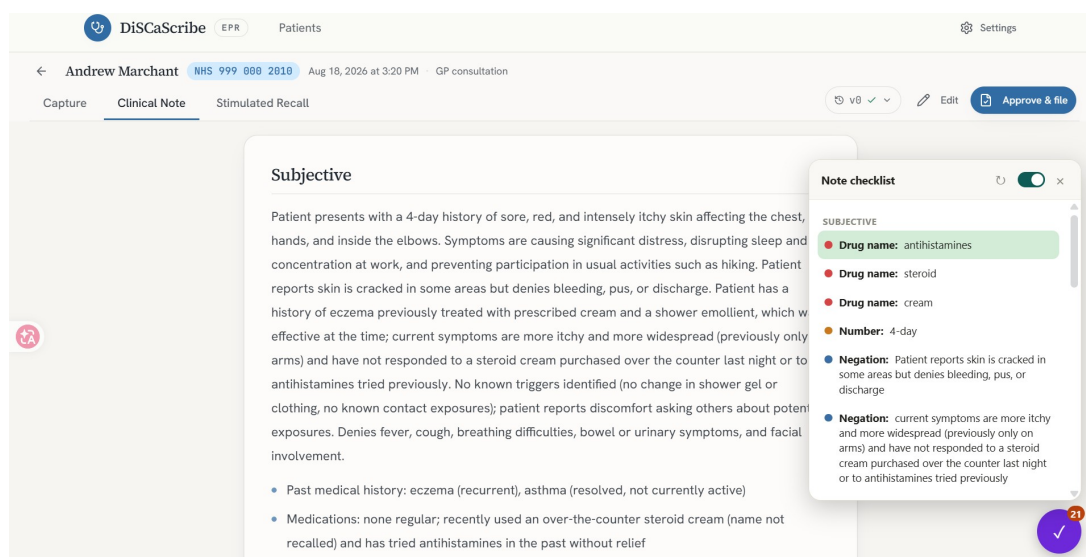
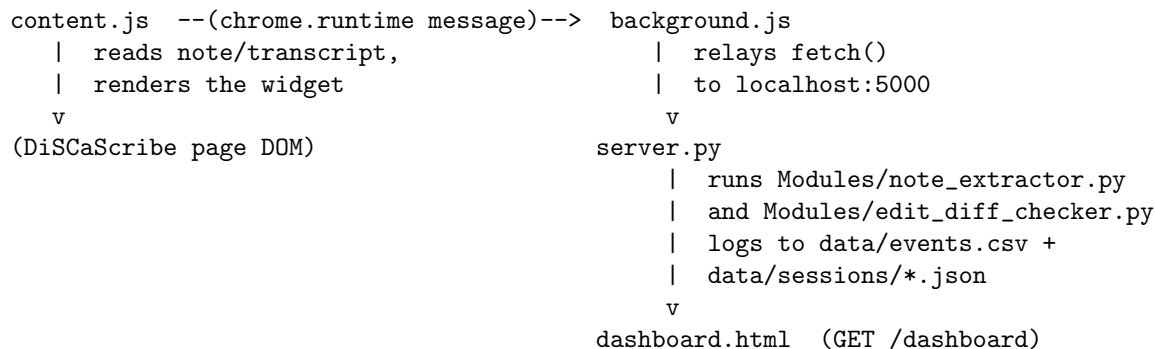


Figure 21: Preview of the google extension

18.4 How the extension connects to the server

- **Host permissions.** `manifest.json` lists both <https://discascribe.vercel.app/> (so `content.js` can be injected) and <http://localhost:5000/> (so the background worker's fetch calls are not blocked by Chrome's own permission model).
- **API key.** Both `extension/background.js` and `server.py` share a static development key, `dev-only-key-replace-me` (server-side, overridable via the `CAPTURE_API_KEY` environment variable), sent as the `X-API-Key` header on every request. This is adequate for one machine testing against itself and **not** adequate once real patient data is involved as it needs to handle multiple machines at the same time
- **Two endpoints the extension calls**
 - **POST /api/check** posts the four SOAP fields, gets back checklist items.
 - **POST /events** posts before/after note text and the transcript when Save is clicked, gets back a critical/edit-ratio summary.

18.5 Architecture at a glance



18.6 Using the extension

- **The floating bubble** (bottom-right, purple) shows a count of unchecked items. Click it to expand the checklist panel, grouped by SOAP section and colour-coded: red = medicine, orange = number (dose/frequency/duration/vital), blue = negation, purple = inference/impression. [Figure 20](#)
- **Clicking a checklist item** jumps to it: switches to the Clinical Note tab, scrolls to and selects/highlights the matching text, and turns the row green. Items are shown compactly as Type: the flagged text, not the full sentence. A checked item stays green across a routine refresh as long as that exact finding has not changed. If the note has nothing to flag at all (e.g. no note has loaded yet), the panel reads “*No SOAP note detected*” rather than leaving a stale checklist showing.
- **The reset button** (a circular arrow icon) in the panel header re-pulls the checklist from the current note and clears every green check, for a clean slate without waiting for a Save.
- **The on/off toggle** both the one inside the panel header and the one in the toolbar popup, write to the same `chrome.storage.sync` flag; flipping either takes effect on every open tab immediately, no reload needed.
- **Every “Save”** (the edit-mode save action, matched on the button title “*Save as the next note version*” not “Approve & file”) ships the note as it was when Edit mode was entered and the note as just saved, plus the transcript, to the server. The server runs `Modules/edit_diff_checker.py`'s `diff.edit` on the two versions and marks each changed span critical or not: a drug removed, a negation flipped, or a dose/frequency/duration changed → critical; a pure rewording → not critical.
- Going to the dashboard <http://localhost:5000/> all the edit data is shown in a reconfigurable format for what is wanted to be seen. Currently implemented is an example.

18.7 Known caveats

- The API key is a static development placeholder (dev-only-key-replace-me) shared between extension/background.js and server.py's CAPTURE_API_KEY default, fine for one machine testing against itself, not fine for anything spanning multiple devices.
- The “Edit” and “Approve & file” buttons were not inspected directly; only “Save”/“Discard” (matched exactly by button title) and the tab buttons were. This does not block the checklist widget or the Save-triggered capture, both of which are independent of Edit/Approve & file.
- This pipeline is specific to DiSCaScribe page. So if any buttons or html changes the extension will fail.